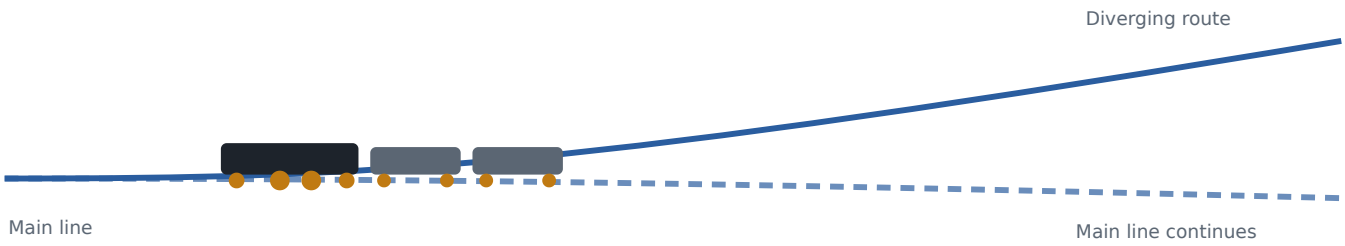
BLENDER 3D TRAIN ANIMATION TOOLKIT

Rail Animator Track It Framework

A concept guide and completion plan: architecture, track and switching systems, vehicles, rigs, consists, the asset workflow, and a worked 2-4-2 steam locomotive with a two-bogie tender.



Subject	“Track It” Blender add-on (`Track it compiled 1.py` , bl_info 1.32, 3,143 lines), plus design sessions 1–6
Sources reviewed	Handoff v8 · the latest add-on source · sessions on track streamlining, curve dragging, nearest-curve selection, switching mechanism, vehicle architecture
Document type	Concept and planning only. Nothing in here has been built or tested in Blender unless it is marked **Built**
Date	26 September 2026

PART 0

How to use this guide

The guide is split into standalone Parts, one per major topic, and each Part is also delivered as its own PDF. Read Parts 1–3 first. After that, jump to whatever you're working on.

The Parts

Part	Topic	What you get from it
0	Introduction	This page, how to read the callouts, glossary.
1	Where the project stands	History from the old Clamp To / shrinkwrap rig to Follow Path. A code audit of the current add-on, feature status, and every issue found.
2	**Curves explained (visual)**	How a Blender curve is built, how Follow Path turns it into motion, what offset_factor really measures, and how Track It adds attach, drag, switching and pivots. Start here if curves feel like a black box.
3	Framework architecture	The four-layer model (Track → Mechanism → Rig → Mesh), the target data model, the per-frame evaluation pipeline, and the proposed package layout.
4	Track system	One-spline rule, arc-table caching, the Track Network graph, overlap zones, joints, banking, and the Track Builder and Track Doctor tools.
5	Switching and route history	How influence switching works now, why two history systems are one too many, and the proposed **Turnout-state + Route Plan** model.
6	Vehicles and pivots	Pivot roles, root placement, body orientation, pivots crossing curves, binding a rig to pivots, and the Vehicle Definition.
7	Motion, drive and wheels	The distance-driven motion model, drive controls, wheel modes (Auto / Direct / Slip), secondary motion (sway, hunting, suspension), and render-safe evaluation.
8	Consist system	Cars following a lead, coupling and uncoupling, driving from any car, and reversing into a siding.
9	Asset workflow	Step by step: turning a plain unrigged model into a Track It vehicle asset in the library.
10	Concept study: 2-4-2 + tender	Rigging an unrigged 2-4-2 steam locomotive with a two-bogie tender and bringing it into the framework.

Part	Topic	What you get from it
11	Roadmap	Phased completion plan, prioritised backlog, testing strategy, and the decisions still open.

Reading the callouts

CURRENT STATE
What the add-on does *today*, taken from the source file and handoff v8.

PROPOSAL
A new idea or improvement from this review. Not built yet.

ISSUE / RISK
A bug or fragile spot found in the code, or a known Blender pitfall (the modal-transform crash, for example).

DECISION NEEDED
A design choice that is yours to make before building.

RIG LAYER
Violet marks the artist-facing control rig: armatures, bones, and the drivers the animator touches.

Diagrams use one colour key throughout: **blue = track / curves / mechanism**, **amber = pivots**, **violet = rig**, **green = proposed / new**, **red = problem**.

Glossary

Term	Meaning in this project
Track / curve / spline	A Blender Curve object holding exactly one spline that a vehicle can ride. Follow Path only ever reads `splines[0]`.
offset_factor	The Follow Path value (0–1, with `use_fixed_location`) that places an object along a curve. It equals *distance ÷ curve length*: it is arc-length based.
Arc table	The add-on's 5,000-sample lookup of cumulative length vs world position, used to convert a world position into an offset_factor.
Held constraint	A Follow Path constraint the object keeps for a curve it has been near. Influence 1 = driving, 0 = inert but kept (its keyframes stay intact).
Overlap zone	A stretch where two curves run on top of each other, so a vehicle can change curve without a visible pop.

Term	Meaning in this project
Divergence point	Where two overlapping curves separate. After it, switching between them is physically impossible.
Joint	End-to-end connection between two curves with no choice (one way on).
Turnout / junction	A real fork where a decision is needed. Proposed as its own object with a keyframeable state (Part 4).
Vehicle root	The object the add-on actually drives along the track. The rig and mesh ride on it and are never bolted to it permanently.
Pivot	A point of the vehicle that tracks the rail on its own, such as a bogie, pony truck or trailing truck. Currently an auto-managed Empty with its own Follow Path.
Consist	A train: a lead vehicle plus coupled cars, each a vehicle with its own pivots.
Route distance / odometer	The running distance (metres) a vehicle has travelled along its actual route, across curve changes. The basis of the route-history log.
Live switching	Changing curves while dragging, with no keyframe written. Only at recorded forks or at a single unambiguous continuation.
Drag Slider	The spring-centred N-panel slider that nudges offset_factor. It is the primary live control; Mouse Drag is deprecated.
2-4-2	Whyte notation: 2 leading wheels (1 axle), 4 driving wheels (2 axles), 2 trailing wheels (1 axle).

THE RULE EVERY PART RESPECTS

Never create or write constraints on an object from a background handler while a 3D transform (`TRANSFORM_OT_translate`) is modal. This caused three identical crashes inside `DEG_id_tag_update → recalcData_objects → transform_modal`. It is why Mouse Drag is deprecated and why live control goes through N-panel widgets.

PART 1

Where the project stands

How the project got from a hand-wired armature rig to a Follow Path framework, what the add-on does today, and what the code audit found.

1.1 From the legacy rig to Track It
1.2 What the add-on can do today
1.3 How the current code is wired
1.4 Handoff v8 vs the latest source
1.5 Code audit: issues found
1.6 What “finished” means

1.1 From the legacy rig to Track It

The original train rigs were built years ago through trial and error, and they worked. But every train was hard-wired to one track by name, and the motion came from a chain of indirect constraints:

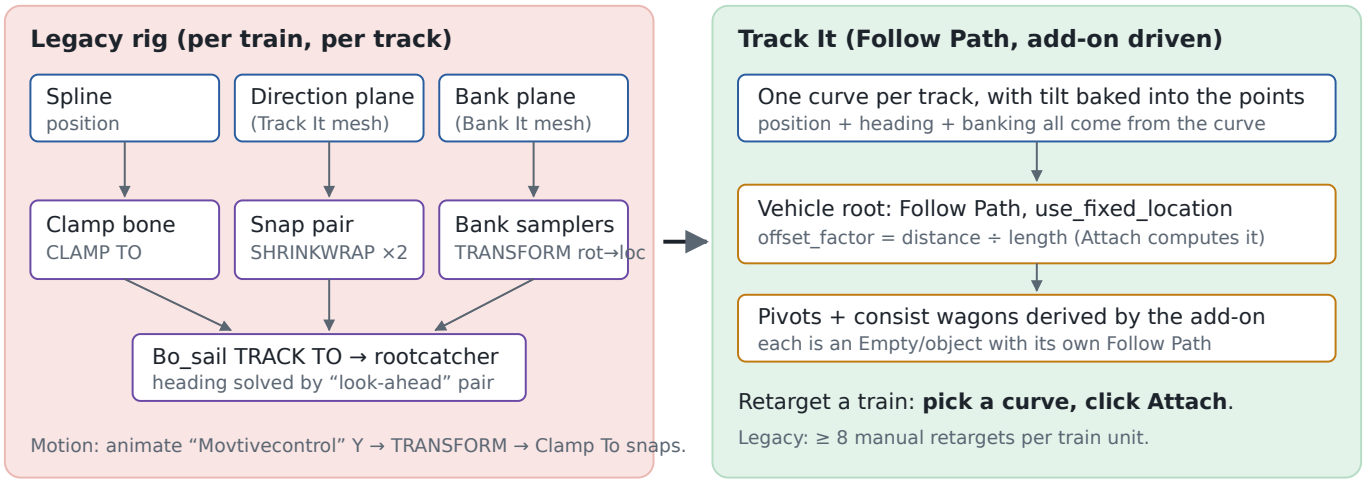


Figure 1.1 Left: the legacy rig reconstructed in session 1 (the “running engine 1” locomotive). Right: the architecture that replaced it.

Milestones so far

When	What changed
May 2026	Legacy rig analysed. Wish list set: attach where the object stands, fewer retargets, track switching.
4–5 Sep	Follow Path found to give position, heading and banking from one curve. Attach operator: closest point → arc-length offset_factor, then zero the local location.
7 Sep	Nearby Curves list (nearest point on curve). Drag Along Curve via proxy Empty, Drag Power, keyframe toggle. Detach mutes instead of deleting. Rotation reset on Attach.

When	What changed
8–10 Sep	Status text, Switching panel, proximity detection with Enter/Exit range, switch keyframing, future-keyframe cleanup. Drag Slider replaces Mouse Drag as the main control. Three modal-transform crashes lead to a standing rule.
11–14 Sep	Vehicle panel with pivot list, auto-created pivot Empties on their own Follow Path, Pivot Data stub. Mute/enabled replaced by influence. Live switching, markers, EXTREME/BREAKDOWN keyframe types, cascade Remove Switch, auto-switch reconstruction.
After v8	Consist panel with wagons, cumulative route distance, route-history log, junction solving, wagons deciding forks themselves, stale-history pruning, Fix Train, Consist Debug box. *(Found in the latest source; not described in handoff v8.)*

1.2 What the add-on can do today

Area	Feature	Status
Track tools	“Track It” / “Bank It” mesh generators (leftovers from the legacy rig)	Legacy
Attach	Attach selected at current world position, curve picker, Nearby Curves list sorted by nearest point	Built
Attach	Detach bakes the pose and sets influence to 0 (constraint and keyframes kept)	Built
Drive	Drag Slider (centre 0.5, springs back) + Drag Power	Built
Drive	Mouse Drag (proxy Empty + G)	Deprecated
Drive	Drag Pad (2D N-panel pad, crash-safe Mouse Drag replacement)	Prototype, not in file
Keyframes	Keyframe all held constraints (offset + influence), context label, Remove Switch cascade with count dialog	Built
Switching	Candidate discovery by proximity, Enter/Exit hysteresis, Switch button, HUD notice, marker + EXTREME key	Built
Switching	Live switching (recorded fork, or unique continuation at a curve end), behind a checkbox	Behind toggle
Switching	Auto-switch reconstruction (BREAKDOWN key from stashed crossing)	Built
Vehicle	Pivot list: auto Empties, offset in metres, same-curve Follow Path	Built
Vehicle	Pivots crossing onto a different curve than the root	Not built
Vehicle	Rig bone / object binding (Pivot Data fields)	Stored, inert

Area	Feature	Status
Consist	Wagon list, coupling distances, route-distance history, wagons crossing forks and deciding forks when leading	Prototype
Consist	Vehicle library, coupling/uncoupling, driving from any car	Not built
Wheels / rig	Auto wheel rotation, Direct mode, slip, sway, hunting, suspension	Concept only
Assets	Vehicle definition, asset library, validation	Concept only

1.3 How the current code is wired

The add-on is one 3,143-line file. It has three kinds of moving parts: **operators** the user clicks, **handlers** Blender calls on every update or frame change, and **data** stored on objects and the scene. The handlers do most of the work.

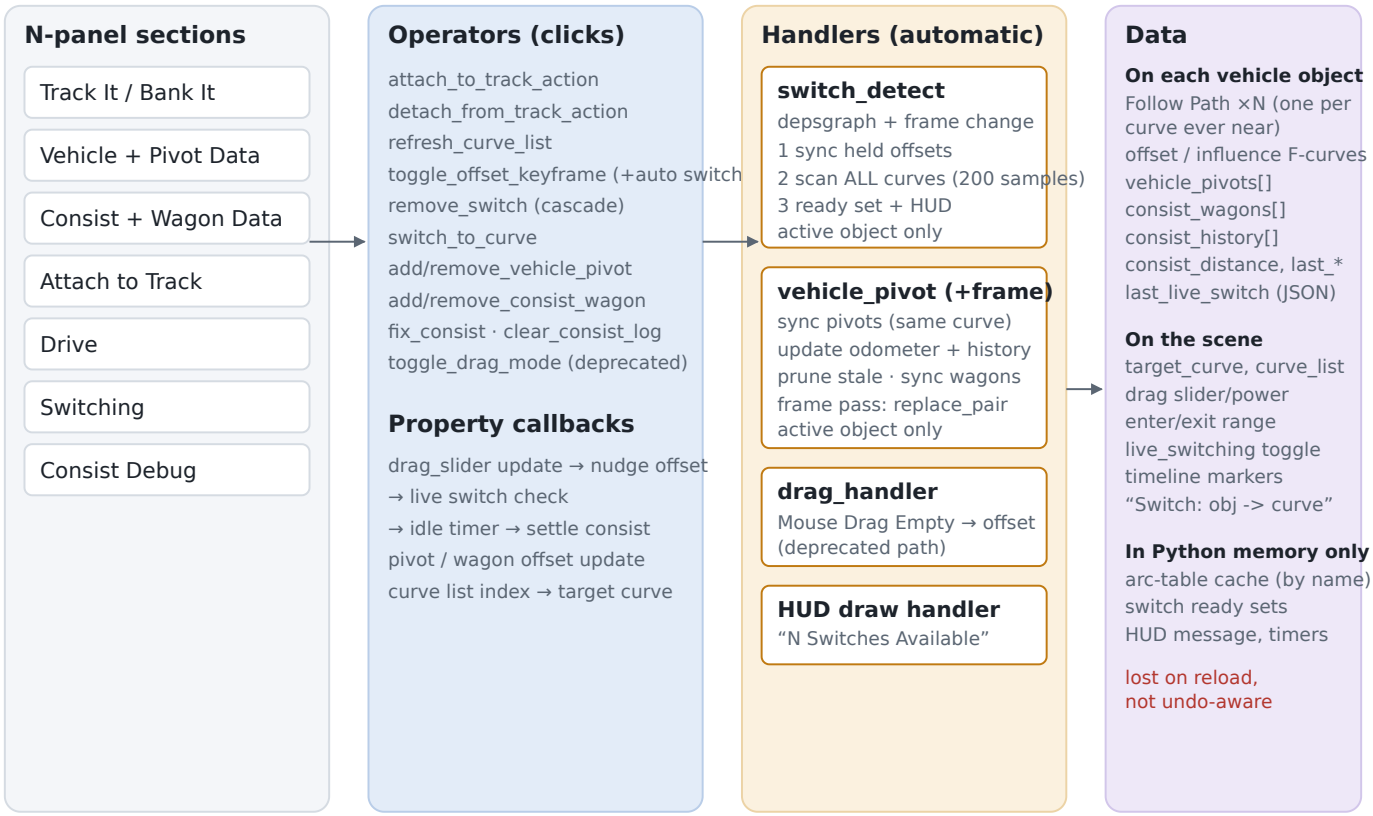


Figure 1.2 Wiring map of `Track it compiled 1.py`. Both main handlers act only on the *active* object, which is behind the render and playback issues listed in 1.5.

1.4 Handoff v8 vs the latest source

RECONCILING THE TWO SOURCES

- The latest source **goes beyond handoff v8**. It contains a full consist prototype: wagons, `TrackIt_ConsistHistoryEntry`, a cumulative route-distance odometer, `_track_it_find_junction`, `_track_it_resolve_route_position`, stale-history pruning, Fix Train, and the Consist Debug box. That is v8's Section 8.2/8.3 "theory" partly built for *wagons*, but not yet for *pivots*.
- The **Drag Pad** prototype v8 describes (`_track_it_draw_drag_pad`, `OT_DragPadGrab_Action`) is **not in this file**. It is probably in the backed-up fork. Worth recovering before Phase 2 (Part 11).
- `bl_info` still says version **(1, 32)** and Blender **(3, 0, 0)**. The code uses the layered-Action API (`action.layers → strips → channelbag`), which only exists in recent Blender (the project runs 5.1). The minimum version should be raised so older Blenders refuse to load it cleanly.
- A **v9 handoff** should record the consist work. This guide can stand in until then.

1.5 Code audit: issues found

Line numbers refer to the latest `Track it compiled 1.py`. Severity: High gives wrong results or breaks renders, Med is fragile or slow, Low is tidy-up.

#	Sev.	Issue	Why it matters / fix direction
1	High	**Syncing only happens for the active object.** Both handlers start with `view_layer.objects.active` (L1171, L2361).	Pivots, wagons and held-offset sync freeze whenever a different object is selected: during playback, during a render, or when a second train exists. Fix: evaluate every registered vehicle, not the selection (Part 7, render-safe evaluation).
2	High	**Arc-table cache never invalidates.** `_track_it_switch_arc_cache` is keyed by curve *name* and only cleared on unregister (L1036–1046, L3111).	Edit, extend or move a curve after it has been used and every offset computed against it is silently wrong until the add-on reloads. Renaming a curve can also hit a stale entry. Fix: key by object pointer plus a hash of points, handles, tilt, `matrix_world` and resolution.
3	High	**Pivots still clamp to the root's curve** (L1731). The consist code already solved crossing for wagons but pivots don't use it.	This is v8's §3.3 gap: a pivot past a curve end snaps to the boundary and the wheelbase breaks. Fix: pivots resolve through the same route resolver as wagons (Part 6).

#	Sev.	Issue	Why it matters / fix direction
4	High?	**Wagons clone the root with `root.copy()`** (L2554). Object-level collection properties (`track_it_vehicle_pivots`, `track_it_consist_wagons`, history) are ID properties and are very likely copied too.	A wagon may inherit a wagon list that points at itself and the root's pivot Empties. If the wagon ever becomes active, the handler would sync that inherited list and fight the real root. *Verify in Blender*, then clear those collections on the clone.
5	Med	**Per-tick cost grows with the scene.** Job 2 scans *every* curve at 200 samples on every depsgraph update. `closest_offset_factor` is a 5,000-point brute-force loop.	Fine for a test scene, slow on a real layout. Fix: a precomputed Track Network (Part 4) plus a spatial grid or KD-tree (`mathutils.kdtree`) per curve.
6	Med	**Handlers write RNA inside `depsgraph_update_post`.**	Every write triggers another update pass. Guards (skip if unchanged) keep this finite, but it is a feedback loop. Prefer `frame_change_pre` for animation-driven work and do UI-driven work in the operator or callback itself.
7	Med	**Two history systems.** Keyframe mining (`_find_recorded_switch_point`) for the root, and the odometer log for wagons.	They can disagree. Keyframe mining was already contaminated twice by leftover test keys, and the odometer needs drift heuristics (`replace_pair`, DEDUP_EPS). Fix: one Route Plan (Part 5).
8	Med	**Identity by name strings.** Ready sets store `curve.name`, markers encode `obj.name`, constraints are named `FollowPath_{curve}`.	Renaming a vehicle orphans its switch markers, and renaming a curve breaks constraint lookup names. Fix: a stable UUID custom property on every managed object; match by pointer.
9	Med	**`build_arc_table` ignores cyclic splines** and spreads samples evenly per segment regardless of segment length.	Closed loops (roundy-round layouts) are missing the closing segment. Long segments get fewer samples per metre. Fix: sample the *evaluated* curve (depsgraph polyline Blender itself follows) including the cyclic segment.
10	Med	**Python-only state isn't undo-aware** (ready sets, caches).	After Ctrl+Z the Switching panel can show stale candidates until the next move. Fix: rebuild on `undo_post` / `load_post`.
11	Low	The Drag Slider registers a new timer on every update (L473).	Harmless but wasteful. Check `bpy.app.timers.is_registered` first.
12	Low	Mute/enabled writes still sprinkled through the code after the influence migration.	Add a one-time “Migrate constraints” operator, then delete the dual-state code.
13	Low	“Track It” / “Bank It” operators generate legacy shrinkwrap planes with `bpy.ops` chains.	Nothing in the new architecture needs them. Replace with a Track Builder (Part 4).
14	Low	Single file, globals, no tests.	Split into a package (Part 3 §3.5) and add headless Blender tests (Part 11).

1.6 What “finished” means

Pulling the goals from every session together, the add-on is complete when an artist can:

1. **Build or import track** quickly, with banking and switches, and have the add-on understand how the pieces connect.
2. **Turn any model into a vehicle** (mine cart, subway car, 2-4-2 steam locomotive) through a guided set-up, and save it to a library.
3. **Couple vehicles into a train**, drag it anywhere including backwards through switches, and have every car and every bogie stay on the correct rails.
4. **Animate it** with ordinary keyframes and easing, with wheels, rods and secondary motion following automatically, or take manual control of any wheel for slip or slide.
5. **Render it reliably**, on a farm or in the background, and get the same result every time.

The rest of this guide is the plan for getting there.

PART 2 · VISUAL EXPLAINER

How curves work, and how Track It moves things along them

A picture-first walk through what Blender stores inside a curve, how Follow Path turns it into motion, and exactly what each Track It feature adds on top. Every later Part builds on this one.

2.1 What a curve object contains
2.2 How a Bézier segment is shaped
2.3 Parameter t vs real distance
2.4 What Blender actually follows
2.5 Follow Path, the engine
2.6 Orientation, tilt and banking
2.7 Why objects “jump” on attach
2.8 Direction, point order, extra splines
2.9 Track It's Attach, step by step
2.10 Animating: offset_factor over time
2.11 Many constraints, one active
2.12 Pivots and wagons: derived points
2.13 One frame, start to finish
2.14 Rules of thumb for track curves

2.1 What a curve object contains

A “curve” in Blender is three nested things. The **object** gives it a place in the world, the **curve data** holds the settings, and inside that is a list of **splines**, each a chain of points. Track It only cares about the first spline.



Figure 2.1 The object → data → spline → point nesting. Tested in session 4: sweeping offset_factor 0→1 on a two-spline curve only ever travelled spline 0.

What each Bézier point stores

- `co` : the point's position (in the curve's *local* space)
- `handle_left` , `handle_right` : steer the shape either side
- `tilt` : roll around the track direction. This is your **superelevation**
- `radius` : scales bevel, and scales objects if “Curve Radius” is on (Track It leaves it off)

Bézier vs Poly/NURBS (“paths”)

A **Path** object (Add → Curve → Path) is a NURBS spline. It starts straight, which is why the switching tests used them. Its points have no handles; the shape comes from point weights. The add-on handles both: Bézier gets the cubic formula, the others are treated as straight lines between points, which is only an approximation for NURBS (Part 4).

2.2 How a Bézier segment is shaped

Between two neighbouring points, Blender draws a **cubic Bézier**. It starts at point A heading toward A's right handle, ends at point B arriving from B's left handle, and blends smoothly in between. A single number **t** runs from 0 at A to 1 at B:

```
position(t) = (1−t)³·A  +  3(1−t)²t·A.handle_right  +  3(1−t)t²·B.handle_left  +  t³·B
```

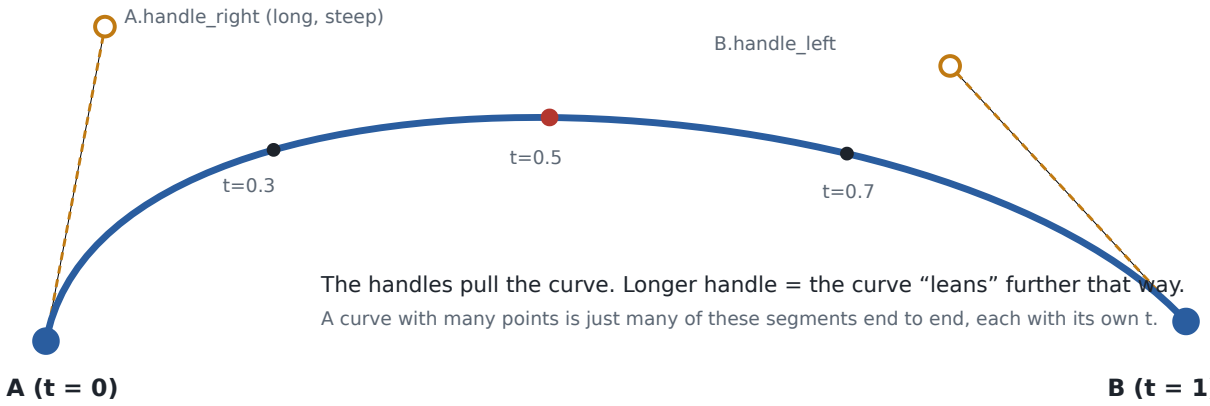


Figure 2.2 One Bézier segment drawn from its four controls, which is exactly what `build_arc_table` evaluates in the add-on.

2.3 Parameter t vs real distance

This is the single most important idea for the whole system. **Equal steps of t are not equal steps of distance.** Where handles are long or points are bunched, t races ahead; elsewhere it crawls. In the segment above, **t = 0.5 is only 46% of the way along**.

Top: 10 equal steps of t Bottom: 10 equal steps of distance (arc length)

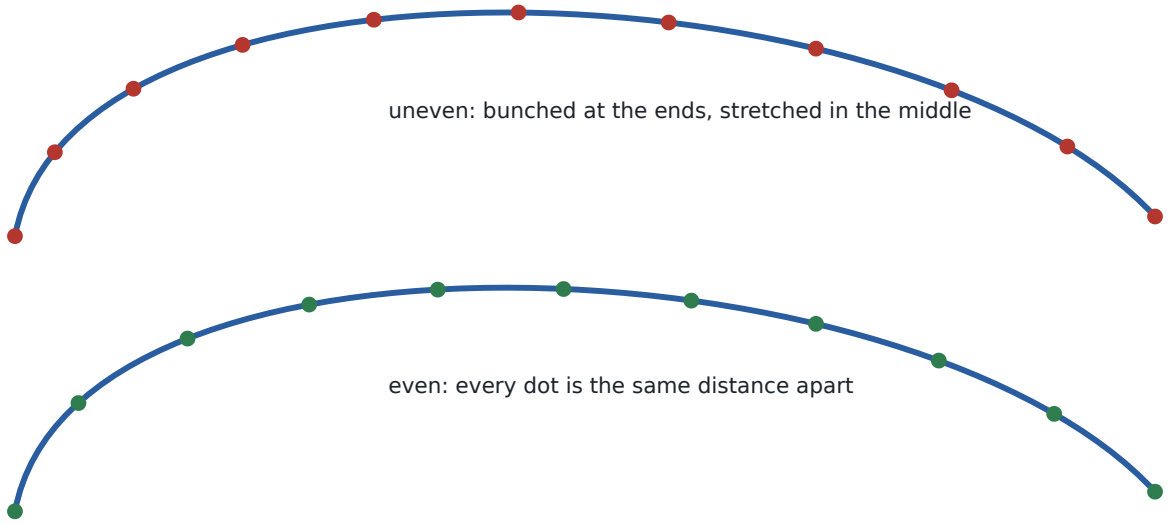


Figure 2.3 Real geometry, computed. A train driven by raw t would speed up and slow down on its own. Follow Path's `offset_factor` uses the green spacing, and so does the add-on's arc table.

CONFIRMED IN THE PROJECT

offset_factor = distance along the curve ÷ total curve length. Setting 0.25 puts the object a quarter of the way along in metres, whatever the handles look like. That is why an offset in metres converts to a factor with one division, `metres / length`, which is exactly what the pivot and consist code does.

2.4 What Blender actually follows

Blender doesn't evaluate the smooth formula at render time. It chops each segment into **`resolution_u`** straight pieces and follows that **polyline**. With a low resolution, a curve's real “followed” shape differs slightly from the smooth one:

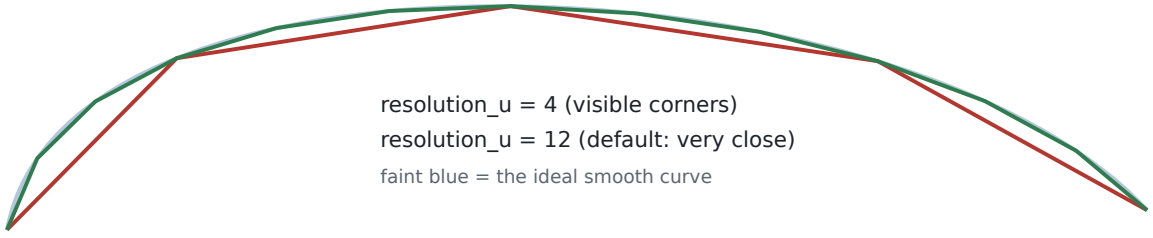


Figure 2.4 Keep track curves at resolution 12 or higher. Proposal (Part 4): build the arc table from the *evaluated* polyline, the same data Follow Path uses, so the add-on and Blender agree exactly.

2.5 Follow Path, the engine

Everything Track It does comes down to setting properties on Blender's own **Follow Path** constraint. The constraint has two modes, and the project chose the second one:

Mode A: “Animate Path” (default)

Position comes from the timeline:

`frame − offset → eval_time / path_duration`

frame 1 — frame 100

✗ offset is measured in **frames**
✗ object always starts at the curve start
✗ moving depends on the playhead

(the “−99.8 offset” Cube.007 was in this mode)

Mode B: Fixed Position ✓ (Track It)

use_fixed_location = True

`offset_factor 0.0 … 1.0 = where it sits`

0.0 — 0.5 — 1.0

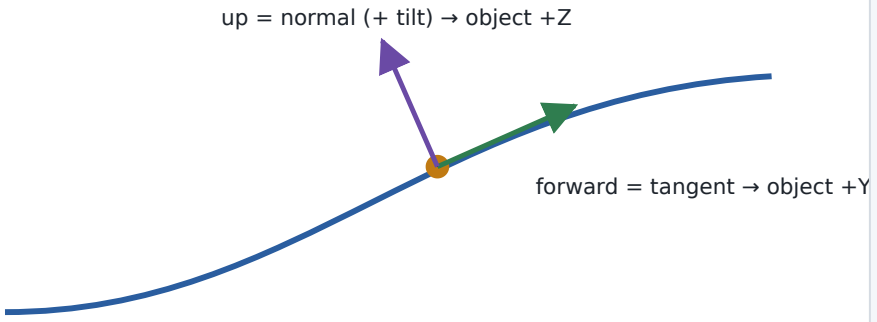
✓ a plain number you can keyframe
✓ can be set to “where it is now”
✓ frame-independent; easing = speed

forward_axis FORWARD_Y · up_axis UP_Z · Follow Curve on

Figure 2.5 The two Follow Path modes. Every constraint Track It creates uses Fixed Position.

2.6 Orientation, tilt and banking

With **Follow Curve** on, the constraint also rotates the object. At the object's point on the curve Blender builds a little coordinate frame: **forward** = the curve's direction there (the tangent), **up** = the curve's normal, then rolled by the point's **tilt**. The object's local **+Y** is aimed forward and its **+Z** up.



Side/top view: the frame travels with the object as offset_factor changes.

Looking down the track



Tilt set on curve points in edit mode (Ctrl+T) = superelevation. It banks the train for free, no bank plane needed.

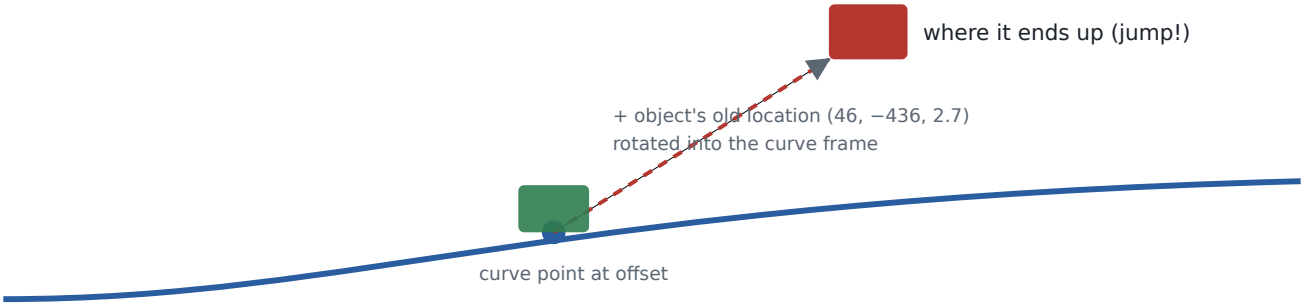
Figure 2.6 This replaced the legacy direction plane and bank plane. The curve itself now carries heading and banking.

WATCH OUT

On curves that climb or twist sharply, the “up” direction can flip or wander (Blender's `twist_method`: Minimum / Tangent / Z-Up). For railways, **Z-Up** is usually the calm choice, since track is mostly flat with gentle grades. Track Doctor (Part 4) should warn when it isn't set.

2.7 Why objects “jump” on attach

The early sessions fought this for a whole day (the “error 1–4” cones). The reason: with Follow Path active, the object's **own** location and rotation don't disappear. They are applied **on top of** the curve point, inside the curve's moving frame.



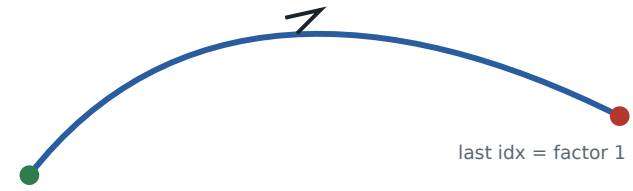
Final position = curve point · curve frame · (object's own local transform)

Fix used by Attach: compute the offset first, then set the object's local location and rotation to zero, so the constraint alone places it.

Figure 2.7 The same rule explains the “re-attach sits wrong” bug (baked rotation left on the object) and the pivot re-attach jump. Both were fixed by resetting to identity every time.

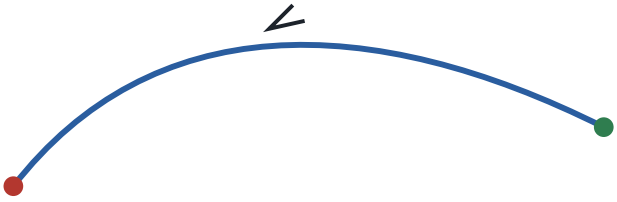
2.8 Direction, point order and extra splines

Point order = direction



Extruding back from the start inserts the new points at index 0.

Switch Direction flips everything



factor f becomes 1 − f. Keyframed offsets would then run backwards. Track Doctor should warn before direction is changed on a used track.

Two practical consequences: a curve's “forward” is set by how it was drawn, and joining two curves (Ctrl+J / F) can leave them facing opposite ways. The Track Network (Part 4) stores, for each connection, whether factor increases or decreases across it. Then direction never has to match.

2.9 Track It's Attach, step by step

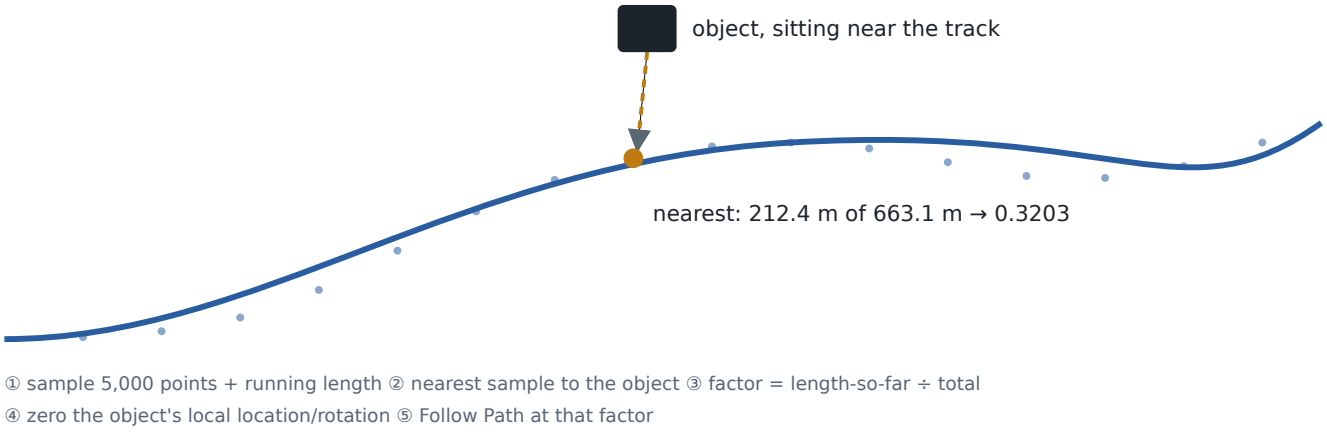


Figure 2.8 What `OT_AttachToTrack_Action` does (numbers illustrative). The same nearest-point math powers Nearby Curves, held-offset sync and Mouse Drag.

Attach also reuses an existing constraint for that curve if the object has been attached before, and sets every other Follow Path to influence 0. That is covered in §2.11.

2.10 Animating: offset_factor over time

Once attached, animating a train means keyframing `offset_factor`. The F-curve in the Graph Editor then reads directly as **position over time**, and its slope is speed:

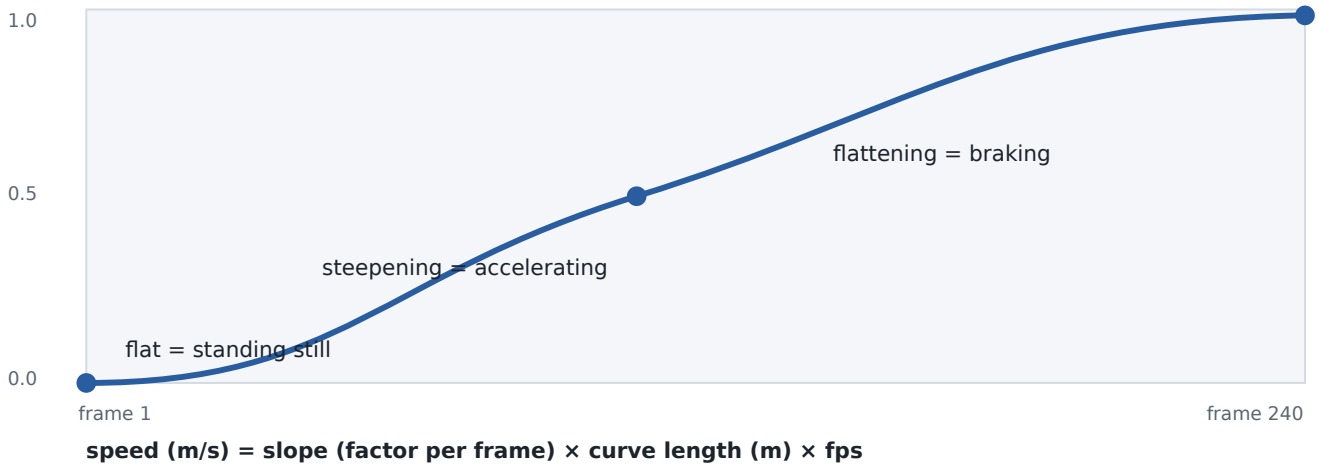


Figure 2.9 One F-curve per held constraint. The catch: the same slope means a different speed on a 100 m curve than on a 2 km one. That is why Part 7 proposes animating **route distance in metres** instead.

The **Drag Slider** is a live way to change that value without keyframing: every slider update nudges offset_factor by `(slider − 0.5) × 2 × Drag Power × Δt`. Nothing is written to the timeline until you press Keyframe.

2.11 Many constraints, one active: how switching works

An object can carry several Follow Path constraints at once, one per curve it has been near. Blender blends constraints by **influence**. Track It keeps exactly one at 1.0 and the rest at 0.0, and switches by swapping which one is 1.0:

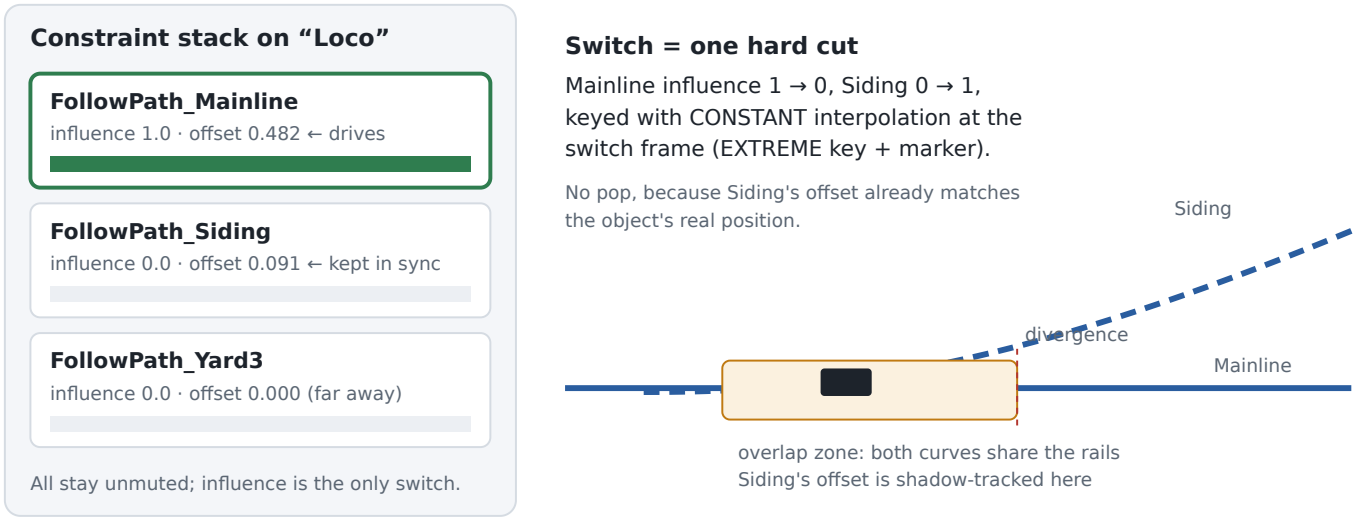


Figure 2.10 Influence switching. Held curves have their offset recomputed every tick from the object's world position (“shadow tracking”), so whichever one is switched to is already in the right place.

2.12 Pivots and wagons: derived points

A pivot (a bogie, pony truck and so on) isn't dragged by the user. The add-on **derives** it from the root: “the same curve, N metres ahead or behind.” Because offset_factor is distance ÷ length, that is one line of arithmetic, and the pivot gets its *own* Follow Path so it takes the curve's heading and tilt at *its* spot:

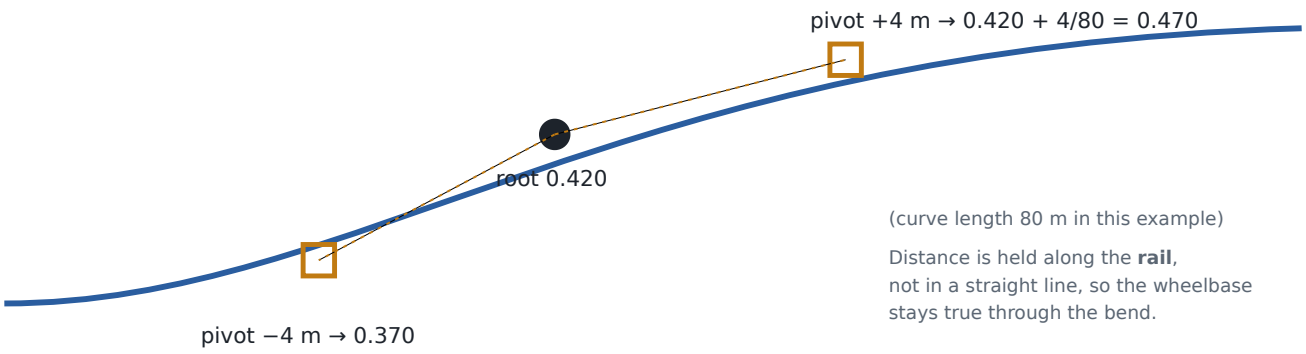
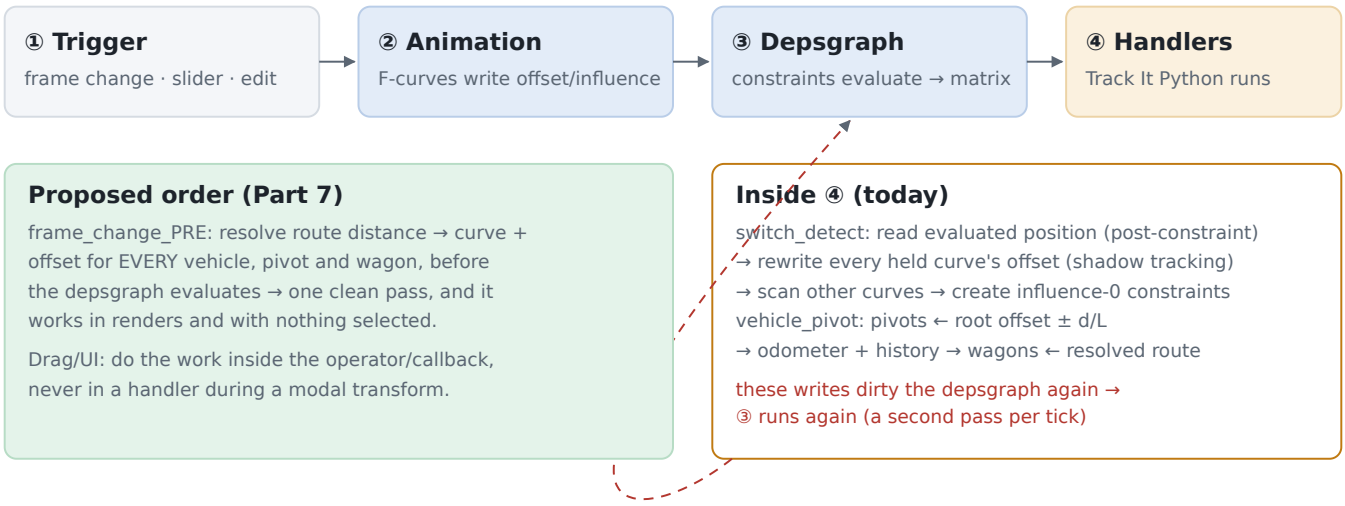


Figure 2.11 `_track_it_sync_vehicle_pivots`. When the result falls past 0 or 1 it is clamped to the end of the root's curve today. Wagons avoid that by looking up the **route history** (Parts 5 and 8).

2.13 One frame, start to finish

When the playhead moves, or you drag the slider, Blender and Track It take turns. Knowing this order explains most “why is it one frame behind” puzzles:



The “one tick stale” bug (matrix_world lagging a fresh offset write) is this ordering. The slider code forces view_layer.update() to get around it.

Figure 2.12 Evaluation order today vs proposed.

2.14 Rules of thumb for track curves

Rule	Why
One spline per curve object	Follow Path reads only `splines[0]`. A branch is a separate object.
Bézier, resolution_u ≥ 12	Blender follows the polyline, so low resolution gives corners (Fig. 2.4).
Twist method Z-Up, banking by tilt	Stable “up”; superelevation is explicit and keyframe-free.
Apply scale on track objects	Non-uniform object scale distorts lengths and tilt. Keep scale (1,1,1).
Real switches need real overlap	The overlap span must be at least as long as the vehicle's longest pivot offset, and longer for trains (Part 5).
Plain joints: endpoints touching	End-to-end pieces with only one way on can be short and modular.
Don't flip direction on used track	Keyframed factors would run backwards (Fig. 2.8).
Keep “Curve Radius” off on constraints	Otherwise the point radius scales the train.

SUMMARY IN ONE SENTENCE

A track is a single spline. Follow Path turns a **0–1 distance fraction** into a position and an orientation. Track It's whole job is to **choose that fraction** (Attach, Drag), **choose which curve's fraction counts** (influence switching), and **derive the fractions of everything that rides along** (pivots, wagons).

PART 3

Framework architecture

The target shape of the finished add-on: four layers with a strict direction of control, a small set of data types, one evaluation pipeline per frame, and a package layout that replaces the single 3,000-line file.

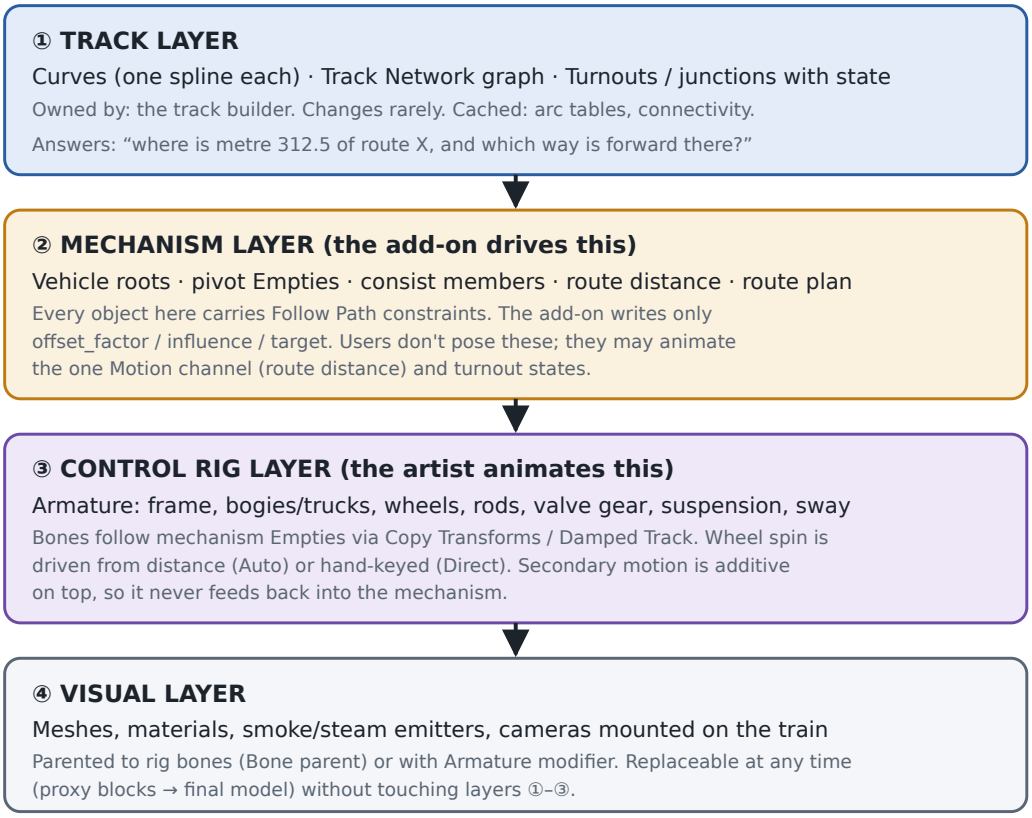
3.1 Design principles
3.2 The four-layer model
3.3 Data model
3.4 Per-frame evaluation pipeline
3.5 Package layout
3.6 Where state lives
3.7 Architecture decisions

3.1 Design principles

Every principle below comes from a decision already made during the sessions. They are written down here so new code can be checked against them.

Principle	In practice
The mesh is never bolted to the mechanism	The add-on moves *roots and pivots*. The rig reads them through constraints, and meshes parent to the rig. Swap a mesh, keep the rig. Swap a rig, keep the mechanism.
Blender does the geometry	Wherever possible the add-on only sets *numbers* (offset_factor, influence, distance) and lets Follow Path, Copy Transforms and Damped Track do the maths. It is inspectable, keyframeable, and banking comes for free.
Distance is the currency	Pivot offsets, coupling gaps, wheel rotation and route history are all real metres. Factors are only ever computed at the last moment, per curve.
Crash-safe live control	No constraint writes from handlers during a modal 3D transform. Live control goes through N-panel widgets (Drag Slider, Drag Pad) or the add-on's own modal operators.
Deterministic playback	The same frame must give the same pose no matter what is selected, whether you scrub forwards or backwards, or whether it's the viewport or a render farm.
No guessing at forks	Curves change only where a decision was recorded or the continuation is unique. Proximity alone never switches a train (the lesson of v8 §4.2).
Everything reversible	Detach keeps constraints, Remove Switch confirms with real counts, and every operator is undoable.

3.2 The four-layer model



Direction of control

Data flows **down only**. Nothing below writes back into a layer above.

What users key

- Route distance (m)
- Turnout states
- Coupled / uncoupled
- Rig controls (wheel mode, slip, sway amount, doors…)

Why it matters

The mine-cart example: the add-on moves the cart along the track; the cart's own rig adds wobble, wheel slip and suspension; the mesh can be swapped any time.

Figure 3.1 The layer stack. The existing add-on implements ① (curves only) and ② (root, pivots, wagons). Layers ③ and ④ are new work (Parts 6, 7, 9, 10).

3.3 Data model

Six types cover the whole system. Blender-side they are `PropertyGroup`s on the scene or on objects, plus a few managed objects. Each carries a stable `uid` string, so renames never break links (audit issue #8).

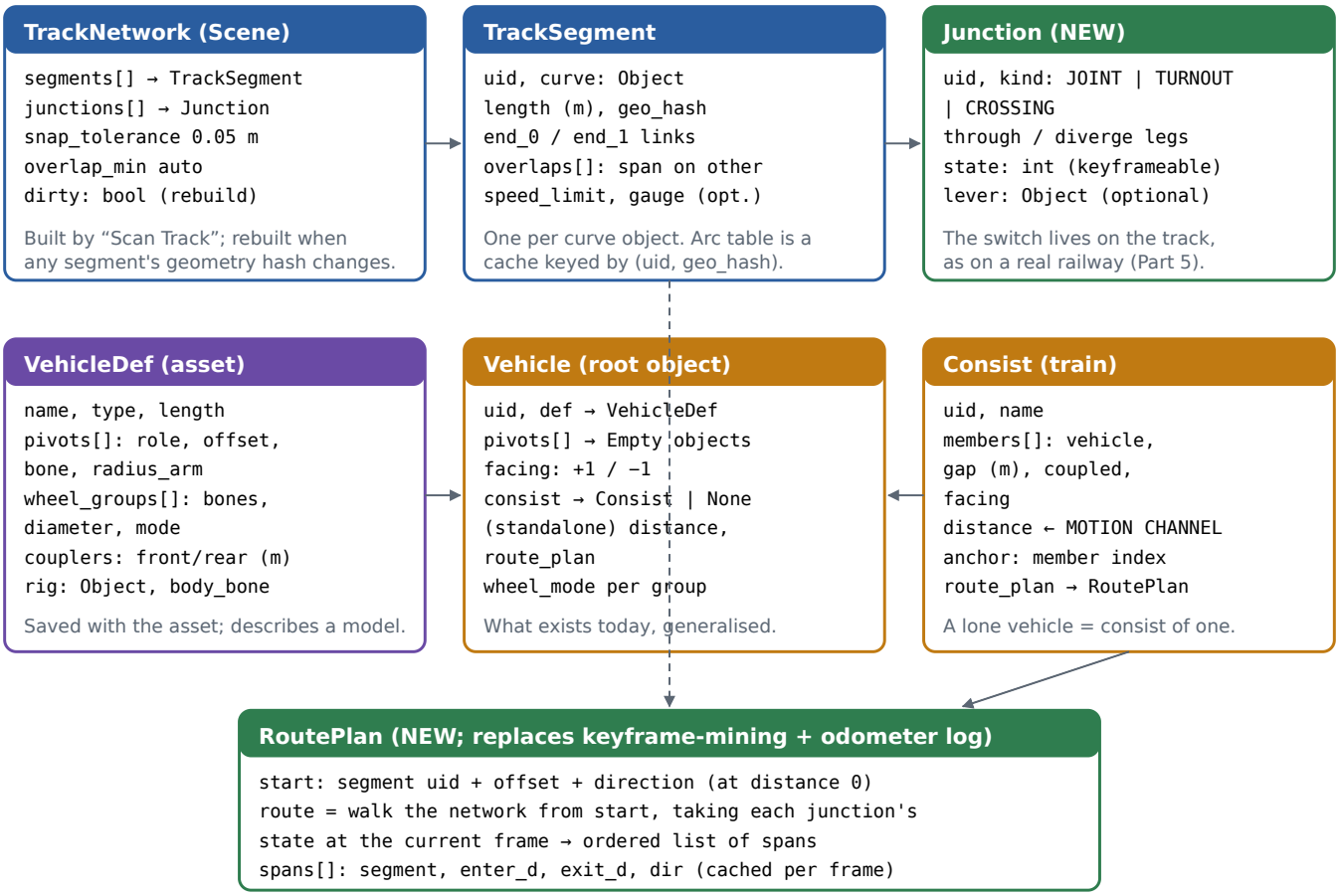
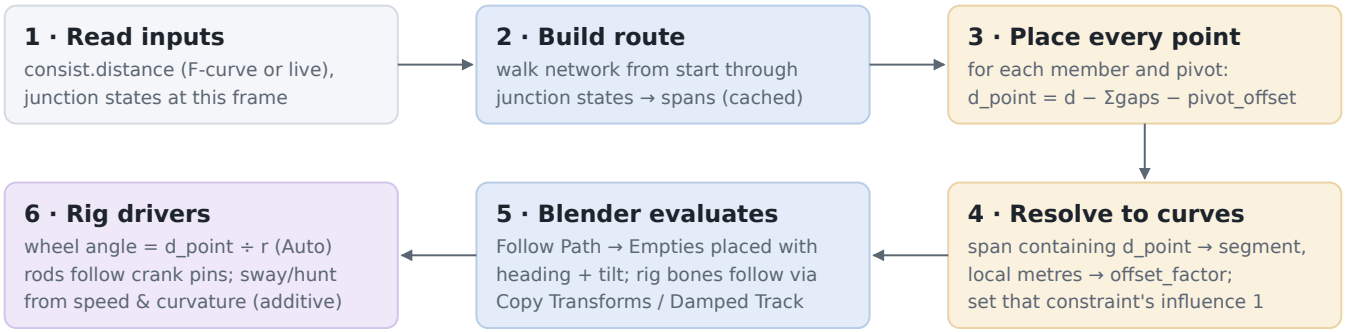


Figure 3.2 Target data model. Green types are new. Amber types exist today in early form (`track_it_vehicle_pivots` , `track_it_consist_wagons` , `track_it_consist_history`).

THE KEY SIMPLIFICATION

Today each vehicle keeps its own history of where it went. In the target model **the train's route is recomputed from two things: where it started, and what the turnouts were set to at each frame**. Anything along the train (root, pivot, wagon, tender bogie) is then “the route at distance *d* − *offset*”. There is no history to go stale, no drift, and no per-wagon fork decisions. Part 5 explains the idea in full; Part 8 applies it to consists.

3.4 Per-frame evaluation pipeline



Where each step runs

Steps 1–4: one Python function, `evaluate_all(scene, frame)`, for **every** consist, called from:

- **frame_change_pre**: playback, scrubbing, and renders (runs before the depsgraph)
- Drag Slider / Drag Pad callbacks: live driving, writes consist.distance first
- operators (Attach, Switch, Couple…), **never** a handler during a modal 3D transform

Steps 5–6: pure Blender (constraints + drivers). No Python, so they evaluate identically in the viewport, in F12 renders and in background (`blender -b`) renders.

Figure 3.3 The target pipeline. The expensive parts (route building, arc tables) are cached and only redone when the inputs change.

RENDER FARMS

Handlers are Python, and some render setups run with scripts disabled (“Auto Run Python Scripts” off) or on machines without the add-on. The pipeline should offer **Bake Train Motion**: write the resolved offset_factor and influence keys for every managed object over a frame range, so the scene plays back with the add-on turned off. See Part 7 §7.8.

3.5 Package layout

Blender loads a folder with `__init__.py` as one add-on (or as an Extension, the Blender 4.2+ format with `blender_manifest.toml`). Splitting by layer keeps each file small and testable:

```
track_it/
  blender_manifest.toml      # extension metadata (replaces bl_info), min Blender 4.4 / 5.x
  __init__.py                # register()/unregister() only
  core/
    arc.py                   # arc tables from the evaluated polyline, cache keyed by (uid,
geo_hash)
    spatial.py               # KD-tree per segment, nearest point/offset queries
    network.py               # TrackNetwork scan: joints, overlaps, turnouts, validation
    route.py                 # RoutePlan walk, distance → (segment, factor, heading)
  mechanism/
    vehicle.py               # root, pivots, facing, constraint management by uid
    consist.py               # members, gaps, coupling, anchor
    evaluate.py              # evaluate_all(): the per-frame pipeline (steps 1–4)
    bake.py                  # Bake Train Motion / Clear Bake
  rig/
    wheels.py                # Auto/Direct/Slip drivers, radius per group
    secondary.py             # sway, hunting, suspension, rail-joint bumps
    bind.py                  # connect rig bones to pivots (Copy Transforms etc.)
  assets/
    vehicle_def.py           # VehicleDef read/write, asset metadata
    wizard.py                # “Make Vehicle” guided setup (Part 9)
  drive/
    slider.py  pad.py        # live controls (Drag Pad = crash-safe mouse drive)
  ui/
    panels.py  lists.py  hud.py  overlays.py
  ops/                       # thin operators that call into the modules above
  handlers.py                # frame_change_pre, load_post, undo_post (rebuild caches)
  tests/                     # headless tests: blender -b -P tests/run.py
```

3.6 Where state lives

State	Stored in	Notes
Track Network, Junctions	Scene PropertyGroup	Saved in the .blend. Rebuilt on demand, or flagged dirty on geometry change.
Junction state	IntProperty (keyframeable)	Constant-interpolation F-curve, which is the switch history. Visible in the Dope Sheet.
Consist distance	FloatProperty (keyframeable)	The one Motion channel. Ease it in the Graph Editor to get acceleration and braking.
Vehicle / pivot / member data	Object PropertyGroups	As today, plus `uid`. Clear these on duplicate (audit #4).
Follow Path constraints	On managed objects	Outputs of the pipeline. Keyframes on them only appear after a Bake.
Arc tables, KD-trees, route spans	Python cache	Rebuilt on `load_post` / `undo_post` / hash change. Never trusted blindly.
HUD message, live-drag timing	Python module globals	UI-only, safe to lose.

3.7 Architecture decisions

Decision	Recommendation	Trade-off
Keep Follow Path, or position Empties directly?	**Keep Follow Path.** It gives banking, is inspectable and bakeable, and matches Blender's own curve sampling.	One constraint per curve per object, managed by uid (that is already the model).
Motion channel: keyed offset_factor or keyed distance?	**Keyed distance on the Consist** (Part 7). Offsets become outputs.	Existing keyframed scenes need a one-time migration operator.
History: keyframe mining, odometer log, or Route Plan?	**Route Plan**: start point + keyframed junction states.	Replaces a lot of working code, so do it in Phase 3, after the Track Network exists.
Evaluate only the active object, or everything?	**Everything**, every frame, in `frame_change_pre`.	Needs the caching work (arc tables, KD-trees) to stay fast.
Single file or package?	**Package / Extension.**	Installation changes from a .py file to a .zip, one time.
Rig: armature per vehicle, or Empties only?	**Armature per vehicle** (Part 6): bones are what animators expect.	A mine cart can use a tiny 3-bone rig. The system doesn't need a complex rig.

PART 4

Track system

Turning a pile of curve objects into a railway the add-on *understands*: accurate, cached geometry; a network that knows which pieces join and where the switches are; banking; and the tools that build and check it all.

4.1 Anatomy of a track piece
4.2 Arc tables done right
4.3 The Track Network
4.4 Turnouts and overlap zones
4.5 Joints and auto-continue
4.6 Banking (cant) tools
4.7 Track Builder and presets
4.8 Track Doctor (validation)

4.1 Anatomy of a track piece

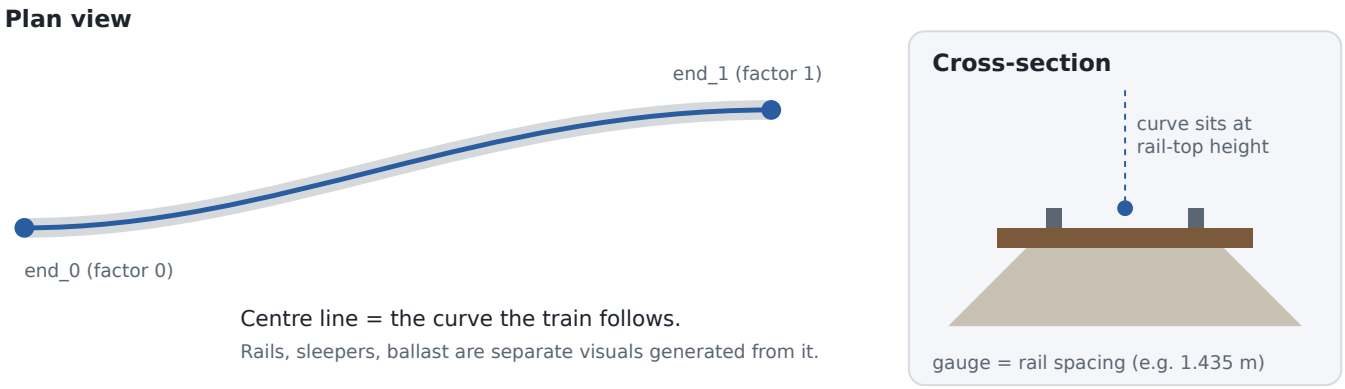


Figure 4.1 Recommended convention: the curve runs along the **centre line at rail-top height**. A vehicle root with origin at rail-top then needs no vertical offset, and pivots land on the rails.

4.2 Arc tables done right

The arc table is the add-on's picture of each curve. Today it is rebuilt from the Bézier formula, ignores cyclic splines, and **is never refreshed after the curve is edited** (audit #2). The fix keeps the same idea and makes it exact and self-maintaining:

1. **Sample the way Blender does**
   Use `mathutils.geometry.interpolate_bezier(knot1, handle1, handle2, knot2, resolution_u + 1)` for each segment. This is the function the first session tried to call with a float. It returns exactly the points Blender's own polyline uses. Include the closing segment when `use_cyclic_u` is on. Transform by `matrix_world`.

2 **Store cumulative length per polyline vertex**
Offsets then map to positions by **interpolating between two vertices**, not snapping to the nearest of 5,000 samples. This is more exact, and the tables are about 20× smaller.

3 **Nearest-point via KD-tree + segment projection**
Build a `mathutils.kdtree.KDTree` of the vertices. For a query, take the few nearest vertices, project onto their adjacent segments, and keep the best. That is O(log n) instead of a 5,000-point loop on every tick.

4 **Key the cache by geometry, not name**
`geo_hash` = hash of point coordinates, handles, tilt, `matrix_world`, `resolution_u` and cyclic flag. Check it cheaply (on `depsgraph_update_post`, only for curves listed in `depsgraph.updates`) and mark dirty on change.



Nothing writes RNA in this path. It only drops Python caches, so it is safe even during a modal transform.

Figure 4.2 Cache invalidation flow.

4.3 The Track Network

Today the add-on discovers relationships *while driving*, by scanning every curve for proximity on every tick. The Track Network does that work **once**, with a **Scan Track** button (and automatically when the geometry changes), and stores the answer as a graph:

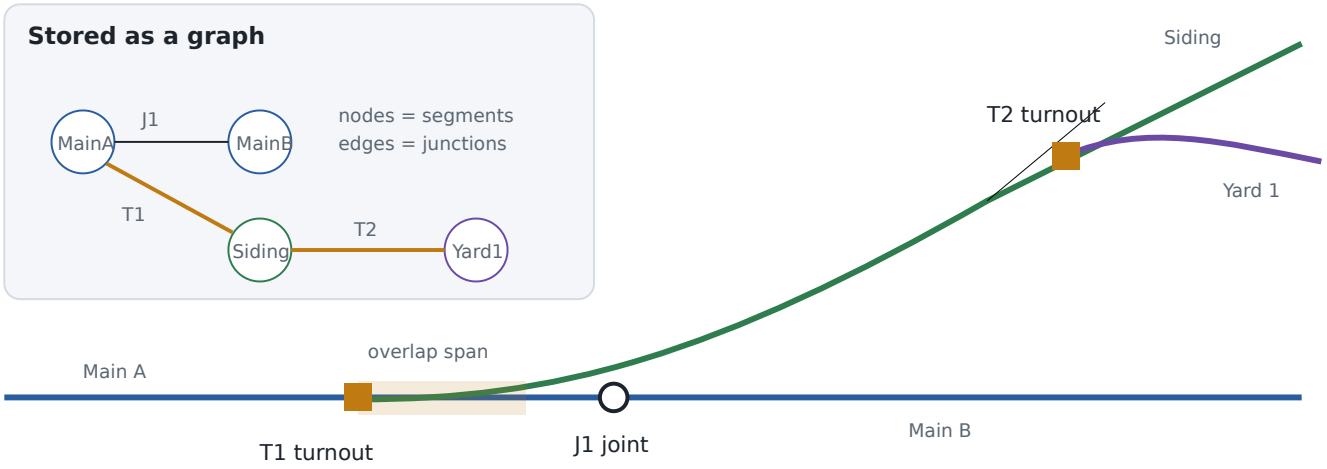


Figure 4.3 A small layout and its network. Each junction records the legs, where on each curve they meet (offset), and whether factor runs the same or opposite way across it.

How Scan Track classifies connections

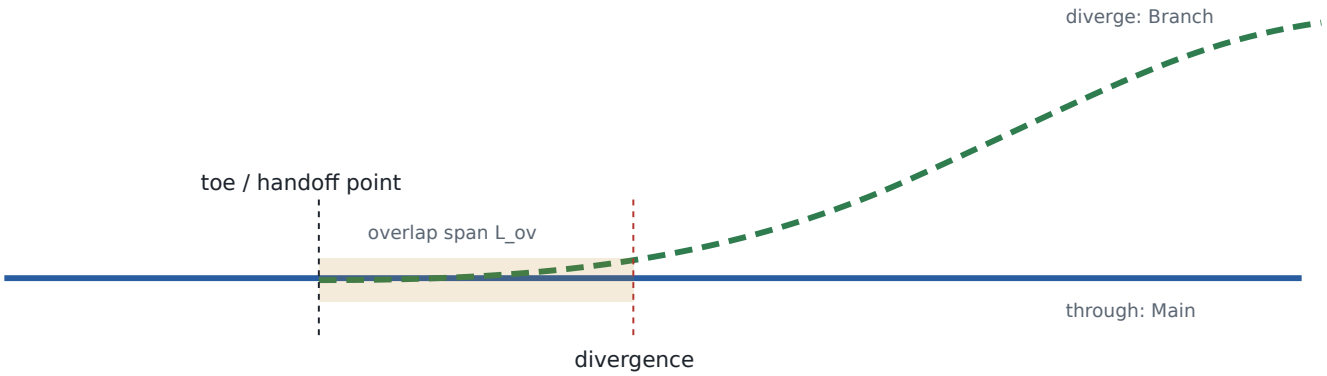
Kind	Detected when…	Stored
Joint	An endpoint of A is within tolerance of an endpoint of B, and the tangents are parallel (either direction).	(A, end), (B, end), direction flip flag
Turnout	An endpoint of B lies on A's *interior*, and B hugs A for a **continuous span** before separating.	A offset range, B offset range, divergence point, span length, which leg is “through”
Crossing / diamond	A and B intersect at an angle, with no shared span.	Informational only: no switching, and a warning if a vehicle could get confused
Dead end	An endpoint with nothing near it.	A buffer stop. Vehicles stop here (factor clamped)

THIS ALSO FIXES V8 §8.5 (YARD FALSE POSITIVE)

The span test in the Turnout row is the “overlap span” fix proposed in v8. Several dead-end sidings that just happen to end near each other fail it, because they don't *hug* each other over a length. They are classified as dead ends, and a train can never jump between them.

4.4 Turnouts and overlap zones

A real turnout has a **toe** (where the diverging route starts), **points** that move, and a **heel/frog** where the routes separate. In curve terms:



Handoff happens at a **single point** where both curves have the same position **and** heading.

Figure 4.4 The anatomy Scan Track looks for. The branch should start **tangent** to the main line at the toe.

How long does the overlap need to be?

Model	Rule
Today (shadow-tracked switching)	L_ov ≥ the vehicle's longest pivot offset, and for trains ≥ the whole consist length, because pivots and wagons clamp to the root's curve (v8 §3.3).
With the Route Plan (Part 5)	L_ov only needs to be **long enough to place the toe accurately** (a few metres). Every point on the train switches when *it* reaches the toe, because the route itself changes curve there. Length stops being a constraint.

DECISION

Should the toe be the **start** of the overlap (natural for turnouts built by duplicating a piece of main line) or an explicit **marker Empty** the user places? Recommendation: auto-detect the start of the overlap, and allow an optional override Empty for odd geometry.

4.5 Joints and auto-continue

A joint is a switch with only one option. In the network model the route **simply continues** across a joint, with no button and no live-switch logic. The requested **“auto-jump to next spline” toggle** becomes a per-junction or scene option:

- **On** (default): the route runs through the joint.
- **Off**: the joint acts as a buffer stop until enabled. This is useful for yard staging, and it can later become a per-vehicle setting (a shunter stops at every joint; an express never does).

4.6 Banking (cant) tools

Banking already works through point tilt. What's missing is **authoring help**. Railways set cant from curve radius and design speed:

```
equilibrium cant angle  θ ≈ atan( v² / (g · R) )        v = design speed (m/s), R = radius (m), g = 9.81
real railways cap cant: ~150 mm on 1.435 m gauge  → θ_max ≈ 6°
```

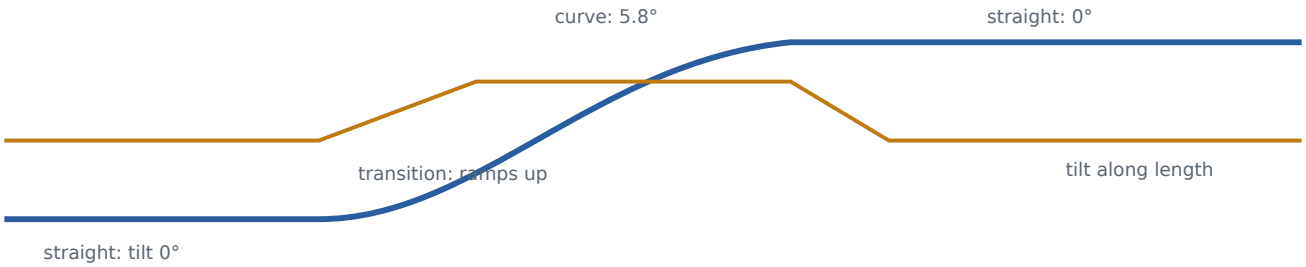


Figure 4.5 The **Auto Cant** operator: measure curvature at each point, apply θ capped at a maximum, and ramp it over transition length so the train rolls in smoothly.

4.7 Track Builder and presets

The legacy “Track It” and “Bank It” buttons build shrinkwrap planes for a rig that no longer exists (audit #13). Replace them with:

Track Builder (Geometry Nodes)

One node-group modifier on the curve: rails (profile swept along the curve), sleepers (instances on points at spacing), ballast. It follows tilt automatically. Parameters: gauge, sleeper spacing, rail profile. It doesn't touch the curve the train rides.

Preset pieces

An asset library of ready track: straights (e.g. 25 m), curves by radius, and **left/right turnouts with a correct toe and overlap already built**. Snap pieces end to end (joints) and Scan Track wires them up. This is the “preset tracks” plan from session 4.

4.8 Track Doctor (validation)

A single button that lists problems, each with a “Select” and, where possible, “Fix” action:

Check	Fix offered
More than one spline in a track curve	Separate into objects
Object scale ≠ 1, or non-uniform	Apply scale
resolution_u below 12	Raise to 12
Twist method not Z-Up	Set Z-Up
Joint gap or tangent mismatch above tolerance	Snap endpoints / align handles
Branch not tangent at the toe	Align first handle to main tangent

Check	Fix offered
Overlap span shorter than the longest vehicle (legacy mode only)	Report which vehicles are affected
Sharp kinks (radius below minimum for vehicles in the scene)	Highlight; suggest smoothing
Direction reversed on a keyed track	Warn before flipping

PART 5

Switching and route history

How the add-on remembers which way a train went. First the current keyframe-and-marker system and its limits, then a simpler model borrowed from real railways: **set the points, and the train follows its own occupied path**.

5.1 How switching works today
5.2 Live switching triggers
5.3 Remove Switch and auto-switch
5.4 Where the current model strains
5.5 Proposal: turnout state + occupied path
5.6 Scenarios walked through
5.7 Mapping today's UI onto it
5.8 Migration path

5.1 How switching works today

CURRENT STATE

Every switch keys **offset_factor and influence on every held constraint** at that frame. The newly active curve's influence key is set to 1 with CONSTANT interpolation, and all others to 0. A timeline marker `Switch: {object} -> {curve}` is added. Manual switches are **EXTREME** keys, reconstructed ones **BREAKDOWN**, and plain position keys are normal **KEYFRAME**s.

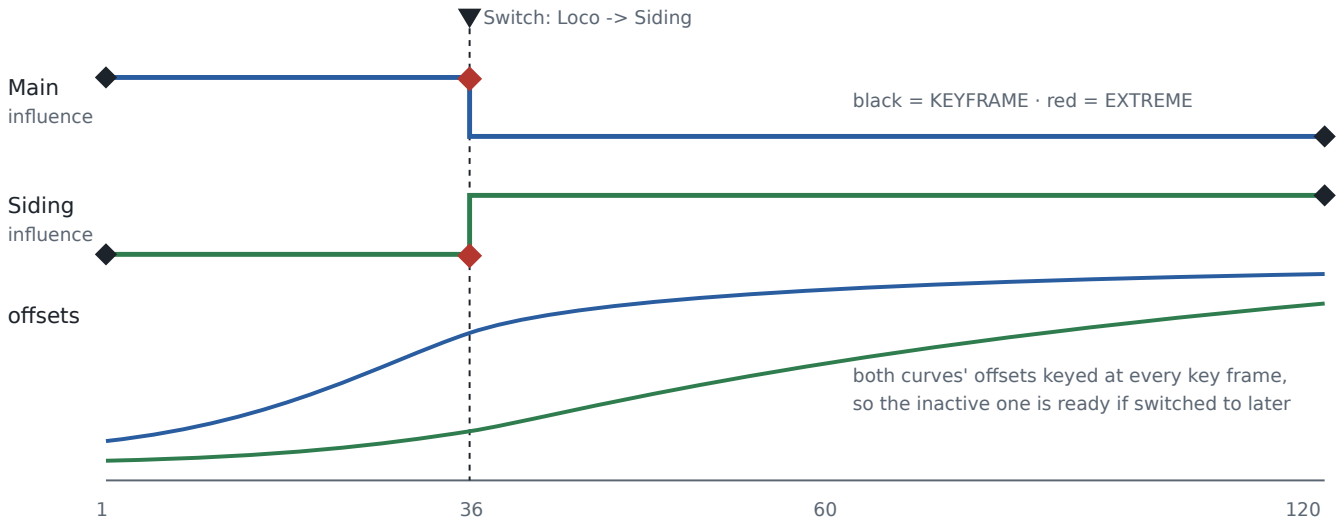


Figure 5.1 A single switch at frame 36, as the add-on records it now.

5.2 Live switching triggers

While dragging (with the Live Switching box ticked), the vehicle changes curve without writing keys, but **only** in two situations. That rule was written after two dead ends (switching at curve ends only, and switching to whatever was nearest):

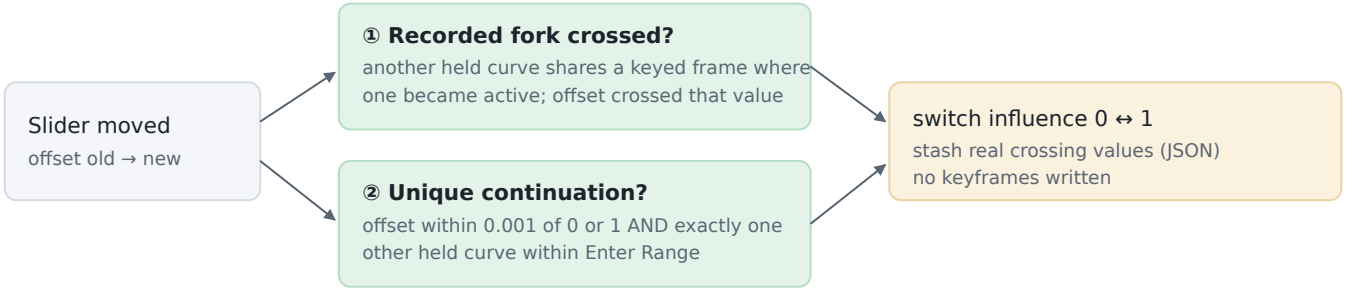


Figure 5.2 `_track_it_try_live_switch_continue`. A dead-zone (1e-4) stops an endless ping-pong at the threshold.

5.3 Remove Switch and auto-switch

Remove Switch (cascade)

On a switch frame, the Keyframe button becomes **Remove Switch**. It deletes this vehicle's switch markers and **every** offset/influence key at and after that frame, hand-placed ones included, after a dialog showing the real counts. Reason: everything later was built on a decision that no longer exists.

Auto-switch reconstruction

If you key the vehicle on a different curve than the last real key recorded (because you live-dragged across a fork), a **BREAKDOWN** switch key is inserted at the midpoint frame. It uses the **stashed real crossing offsets**, not the current resting position, so playback runs smoothly with no pop.

5.4 Where the current model strains

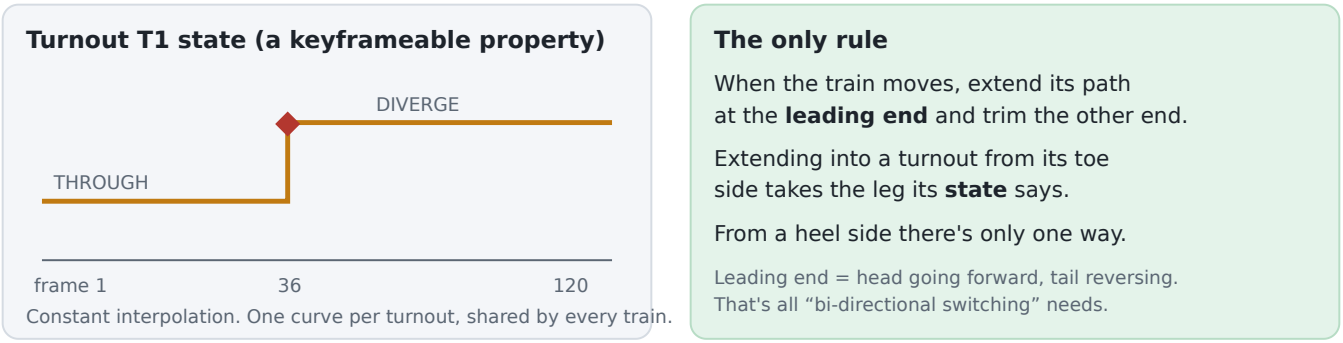
Situation	What happens now
Leftover test keys on a constraint	Keyframe mining reads them as real forks. This has happened twice (v8 §8.3).
Pivots or a trailing car reach a fork before the root	Pivots clamp to the root's curve. Wagons use a separate odometer log with dedupe, “authoritative” overwrite and stale-entry pruning.
Deleting an early switch	Must also delete every later key, because later offsets depend on it.
Two curves meeting at two junctions (loops, reversing)	The 1-D odometer maps two places to one distance. Pruning drops the older claim, which works but needs care.
Switchback (main → curve 2, reverse onto curve 3)	Handled by playhead position. It only works if keys are placed in the right order.
Two trains through the same turnout	Each vehicle mines its own keys, so nothing ties them to one real turnout.

Situation	What happens now
A detour inside a single frame	Can't be reconstructed at all (v8 §8.4).

None of these are coding mistakes. They all come from one choice: **storing the route as per-curve offsets on each vehicle**. Change what is stored, and they go away.

5.5 Proposal: turnout state + occupied path

On a real railway, trains don't choose. **The points are set, and the train goes where the rails lead.** The model below copies that:



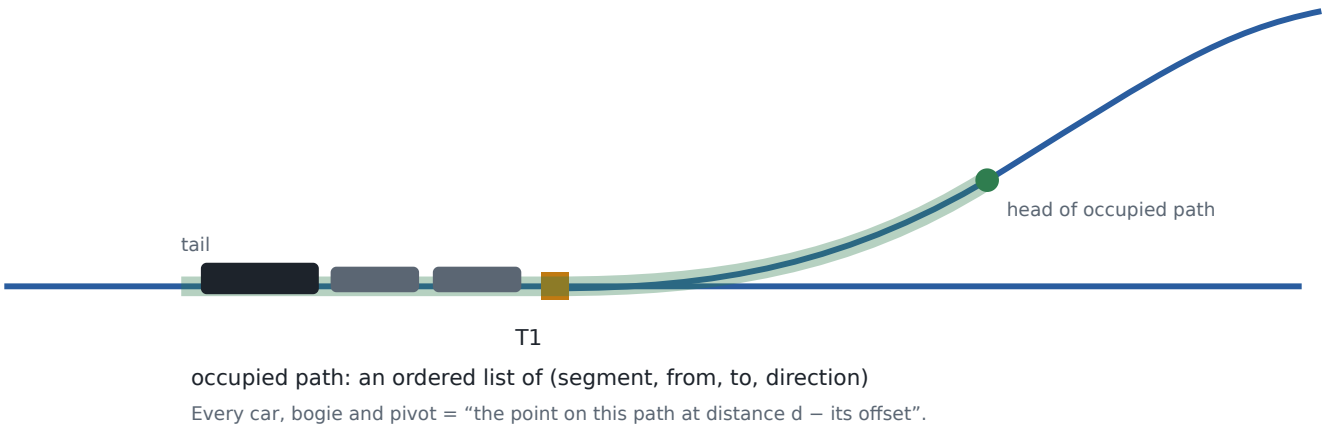


Figure 5.3 The train owns a contiguous **occupied path**. Points ahead of the train are decided only when the leading end reaches them; cars already on a leg stay on it.

Why this is deterministic when scrubbing

The occupied path at frame *F* comes from **replaying** the train's distance curve from its start frame to *F*, extending and trimming as it goes, and reading each turnout's state at the frame the leading end reaches it. That is a small step per frame, **cached per frame** and invalidated only when the distance keys, the turnout keys or the track change. Scrubbing backwards just reads the cache. Rendering frame 200 on a farm replays 1→200 once (milliseconds for normal scenes).

WHAT DISAPPEARS

- No per-curve offset keys on vehicles (offsets become outputs, bakeable).
- No keyframe mining, no shadow-tracking of held curves, no odometer drift or pruning heuristics.
- No cascade delete: removing a turnout key only changes where the train goes *afterwards*. Nothing downstream is invalidated, because distance is independent of route.
- No pivot/wagon special cases: they are all points on the same path.
- No overlap-length rule for consists (Part 4 §4.4).

Interlocking: safety for free

Changing a turnout while part of the train is already over it does nothing to the cars that are already past the toe, because they are on the occupied path. The UI can warn “T1 is occupied” and show the state change taking effect for the *next* train, just like real signalling. This is the gating rule from v8 §8.1 (“if any point has passed the divergence, the fork is no longer a choice”), and it happens automatically.

5.6 Scenarios walked through

Scenario	With turnout state + occupied path
Simple diverge at frame 36	Key T1 = DIVERGE at or before the frame the head reaches it. Key the train's distance as usual. Pivots and wagons each take the branch when *they* reach the toe.
Switchback: main → curve 2, then reverse onto curve 3	T1 (main/curve 2) = DIVERGE. The train runs onto curve 2 past T2. Key T2 = DIVERGE toward curve 3, then key the distance going **down**. The tail is now the leading end, it reaches T2 from its toe side, and takes curve 3. No special case.
Back a train into a siding, decided by the last car	The same thing: when reversing, the last car's end of the path extends through the siding turnout's state. The locomotive follows later on the same path.
Drag live without keys	Dragging changes distance and extends the path using turnout states at the *current frame*. Option: **“Drag uses: current state / next keyed state”**. The second gives the behaviour asked for in session 5 (“if a switch exists, dragging from before frame 36 still takes it”).
Two trains, one turnout	Both read the same T1 F-curve. Set it to DIVERGE for train A at frame 40, back to THROUGH at frame 90 for train B. That is a realistic yard timetable.
Remove a decision	Delete the T1 key. The train now stays on main. Its speed and timing keys still make sense, so nothing else is deleted.
Joint	Always a single way on, so the path just continues (unless the joint's auto-continue is off: buffer stop).

DECISION NEEDED

Per-train overrides? Some users will want “this train always takes the branch at T1, whatever the turnout says” (for example, crowd scenes). Recommendation: add an optional **Route Plan override** list per consist, an ordered list of (junction, leg) choices tried before the turnout state. It is off by default so there is one source of truth.

5.7 Mapping today's UI onto it

Today	Becomes
Switching panel: “Ready to Switch → X”	“Junctions ahead”: the next N junctions within look-ahead distance of the leading end, each with a THROUGH / DIVERGE toggle and a key button. The HUD “Switch Available” notice fires when the leading end is within range of an unset turnout.
Switch button (EXTREME key + marker)	Keys the turnout state (EXTREME type kept for visibility). The marker becomes `T1 → Siding` by junction uid, so renames are safe.
Keyframe Offset Factor	**Key Position**: keys consist distance. The label logic stays the same (key or remove).
Remove Switch cascade dialog	Plain “Remove turnout key”. The dialog can list which trains' routes will change.
Live Switching checkbox	Not needed. Live dragging always follows the network.
Enter/Exit Range	Only used by Scan Track (tolerances) and the HUD look-ahead.

5.8 Migration path

1. **Build the Track Network (Part 4)**
   It is useful on its own: it speeds up today's candidate discovery and fixes the yard false positive.

2. **Add Junction objects + state, alongside the old system**
   The Switch button writes both the old keys and the turnout key, so nothing breaks.

3. **Implement the occupied-path evaluator behind a toggle**
   Apply it to consists first, where the gains are biggest, then to pivots.

4. **Convert existing scenes**
   A **Convert Keyed Switches** operator reads each vehicle's EXTREME/BREAKDOWN keys and markers, creates turnout keys, and converts offset keys to a distance curve (integrating length along the resolved route).

5 **Retire the old path**
Remove keyframe mining, shadow tracking, stash JSON and the odometer log once the converter has been checked on the test scenes.

PART 6

Vehicles and pivots

What a “vehicle” is in data terms. It is not a list of N identical pivots but a few distinct **roles**, together with how the body gets its position and angle, and how a rig plugs into the mechanism.

6.1 Vehicle families
6.2 Pivot roles
6.3 Where the root goes
6.4 Body orientation and the chord effect
6.5 Pivots crossing curves
6.6 Binding a rig to the pivots
6.7 Facing and running backwards
6.8 The Vehicle Definition

6.1 Vehicle families

Family	Track contact	Mechanism needed
Mine cart / tram with one truck	One short rigid wheelbase	Root only (or two axle points if the wheelbase is visible on curves)
Four-wheel wagon	Two axles fixed to the frame	Two **axle points**; the frame aims between them
Bogie coach / subway car	Two swivelling bogies	Two **tracked pivots**; the body rides on both
Steam locomotive (e.g. 2-4-2)	Rigid driving wheels + swinging pony/trailing trucks	Driver **axle points** set the frame; **radial** trucks swing about pins on the frame
Tender (two bogies)	Two bogies	Same as a bogie coach, coupled to the loco by a drawbar
Articulated (Mallet, Garratt, articulated tram)	Several frames hinged together	Several bodies, each a sub-vehicle. A consist with zero gap and a hinge

6.2 Pivot roles

The v8 pivot list treats every pivot the same (“offset in metres, own Follow Path”). That is correct for bogies but not enough for locomotives. The vehicle-architecture session already identified three roles, and adding a fourth covers everything:

One vehicle can mix all four roles

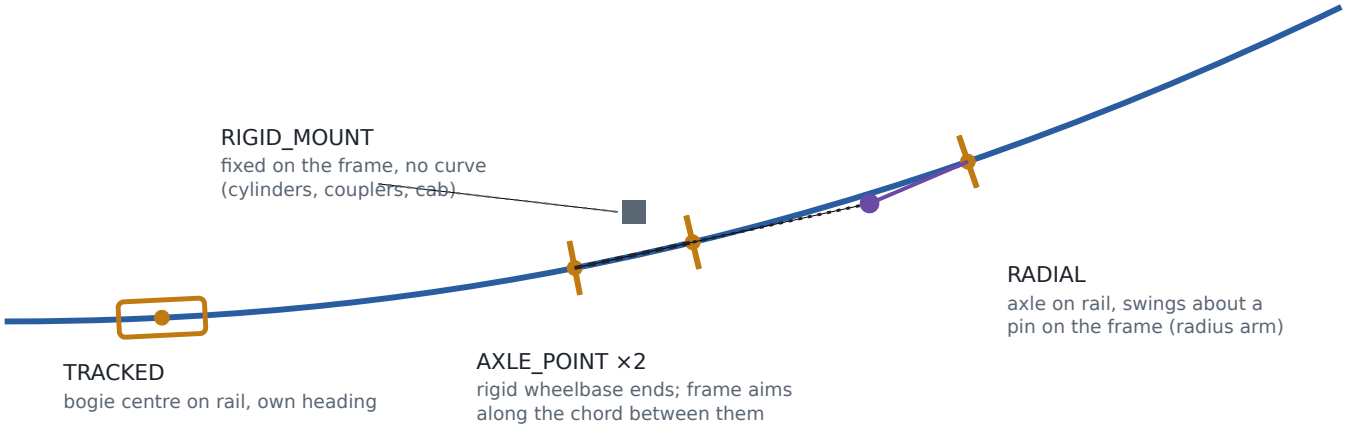


Figure 6.1 Pivot roles. Only TRACKED, AXLE_POINT and RADIAL sample the rail. RIGID_MOUNT is just a bone on the frame.

Role	Mechanism object	How its transform is found
TRACKED	Empty + Follow Path (as today)	Route point at *d + offset*, with that point's heading and tilt.
AXLE_POINT	Empty + Follow Path	Same as TRACKED. Only its **position** is used; the frame's heading comes from the pair.
RADIAL	Empty + Follow Path (axle) + stored `pin_offset`	Axle on rail; the truck bone sits at the axle and **Damped Tracks** toward the pin on the frame.
RIGID_MOUNT	None (bone in the rig)	Child of the frame bone.

6.3 Where the root goes

Option A: root at the vehicle's centre

The root sits midway between the two defining points (bogie centres, or the ends of the rigid wheelbase). Pivot offsets are symmetric (±). It looks natural in the viewport and matches how most models are built. Coupler positions are stored as `front_coupler` / `rear_coupler` distances in the definition.

Option B: root at the front coupler

Consist maths becomes one running number, but every pivot offset is negative and the gizmo sits at the nose. It is awkward to author, and flipping the vehicle's facing moves the reference point.

RECOMMENDATION

Option A (centre), plus coupler distances in the definition. The consist code computes spacing as `rear_coupler(car n) + gap + front_coupler(car n+1)`, and the user only enters the gap (often 0 when couplers touch). That keeps the “front-referenced chain” idea from session 4 without making the vehicle awkward to author.

6.4 Body orientation and the chord effect

A long car on a curve does **not** hug the rail: its body is a straight line between its two bogies, so its middle sits **inside** the curve and its ends overhang **outside**. Today's root rides the rail itself, so a mesh parented straight to the root clips through the inside of tight curves.

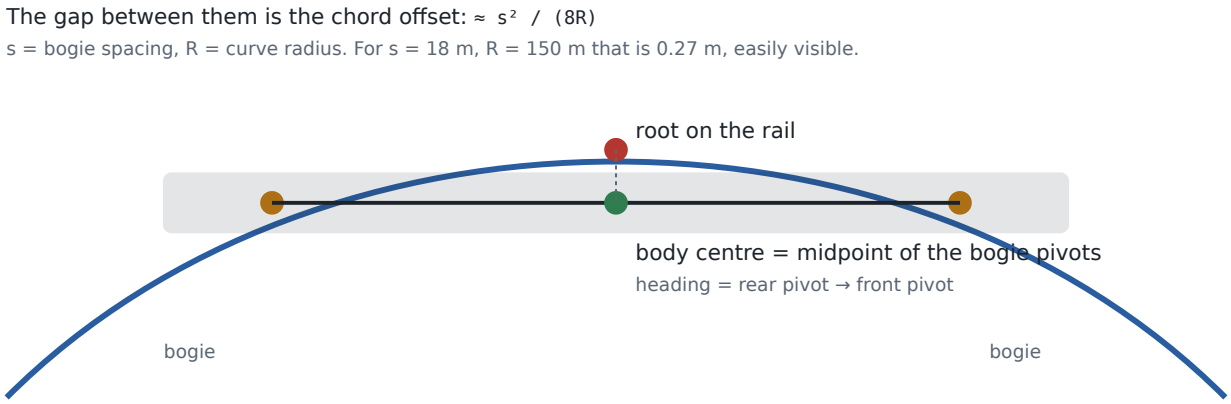


Figure 6.2 The body should come from the **pivots**, not the root. This is also how real rail vehicles behave.

Built with native constraints on the body bone

```
BODY bone
  Copy Location   → Pivot_Front Empty           influence 1.0
  Copy Location   → Pivot_Rear  Empty           influence 0.5   (= midpoint of the two)
  Damped Track    → Pivot_Front Empty  (Y)      heading along the chord
  Copy Rotation   → Pivot_Front (roll only, mix ADD, influence 0.5)
  Copy Rotation   → Pivot_Rear  (roll only, mix ADD, influence 0.5)   averaged banking
```

For a steam loco the same pattern uses the two **driver AXLE_POINTs** instead of bogies (Part 10).

6.5 Pivots crossing curves

The known gap (v8 §3.3): pivots are placed on the root's curve and clamp at its ends. Two ways to close it:

Approach	How	When
Short term: reuse the wagon resolver	Call `_track_it_resolve_route_position(root, root_distance + pivot.offset)` for each pivot, exactly as `_track_it_sync_consist_wagons` already does for wagons, then set that curve's constraint to influence 1 on the pivot Empty.	Now, small change
Target: the occupied path	Pivots are points on the train's occupied path (Part 5). Nothing special is needed.	Phase 3

6.6 Binding a rig to the pivots

The Pivot Data panel already stores `rig_object` and `rig_bone` (inert). A **Bind Rig** operator turns that into constraints on the pose bones, and **Unbind** removes exactly those constraints (tagged by name prefix `TI_`):

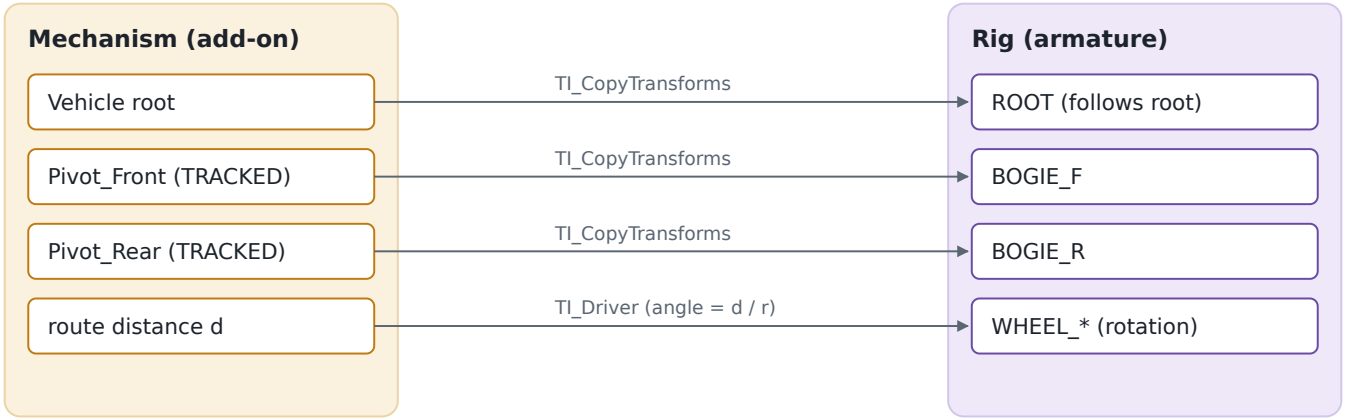


Figure 6.3 Binding is one-way (mechanism → rig) and fully removable. The artist's own constraints and keys on the rig are never touched.

BONE VS OBJECT

Pivot Empties should be **hidden and unselectable** in finished assets (the earlier plan), so nobody grabs them by accident. The Pivot Data panel keeps a “Show mechanism” eye toggle for debugging.

6.7 Facing and running backwards

A locomotive can run tender-first, and a coach can be turned end for end. Add a per-vehicle **facing** (+1 / −1):

- Pivot offsets and coupler ends are multiplied by facing when placing points.
- Follow Path on the root and pivots uses `forward_axis = 'FORWARD_Y'` when facing = +1 and `'TRACK_NEGATIVE_Y'` when −1, so the model turns 180° with no rig change.
- Wheel rotation sign flips automatically, because it is computed from distance × facing.
- This is the “direction flipping system” saved for later in session 5. It is a small addition once pivots are role-based.

6.8 The Vehicle Definition

Everything above that describes the *model* (not its current position) belongs in a definition saved with the asset:

```
VehicleDef "LNER_242T_Proto"
  type: LOCOMOTIVE          length_over_couplers: 11.80 m
  front_coupler: +5.90 m    rear_coupler: -5.90 m      facing_default: +1
  pivots:
    LEAD_TRUCK   role RADIAL      offset +3.95  pin_offset +2.60  bone TRUCK_F
    DRIVER_1     role AXLE_POINT  offset +1.05                    bone AXLE_D1
    DRIVER_2     role AXLE_POINT  offset -1.05                    bone AXLE_D2
    TRAIL_TRUCK  role RADIAL      offset -3.95  pin_offset -2.60  bone TRUCK_R
  frame: aims DRIVER_2 → DRIVER_1, bone FRAME
  wheel_groups:
    drivers   bones [WHL_D1_L, WHL_D1_R, WHL_D2_L, WHL_D2_R]  diameter 1.73 m  mode AUTO
    trucks    bones [WHL_F_L, WHL_F_R, WHL_T_L, WHL_T_R]      diameter 0.97 m  mode AUTO
  secondary: sway 0.4, hunting 0.2, bounce 0.3, joint_spacing 18.3 m
```

Numbers are illustrative (Part 10 works through a real layout). Stored as a JSON string property on the asset's root object, with a matching PropertyGroup for editing in the UI.

PART 7

Motion, drive and wheels

How the animator actually moves a train: one distance channel, friendly timing tools, live drive controls that can't crash Blender, wheels that roll on their own or on command, and secondary motion that makes it feel heavy.

7.1 One Motion channel: distance
7.2 Timing tools
7.3 Live drive controls
7.4 Wheel modes
7.5 Secondary motion
7.6 Speed readout
7.7 Render-safe evaluation
7.8 Bake Train Motion

7.1 One Motion channel: distance

Today the animator keys `offset_factor` on every held constraint. Speed then depends on each curve's length, and a train crossing three curves has three unrelated F-curves. Instead, key **one number per train: distance travelled, in metres**.

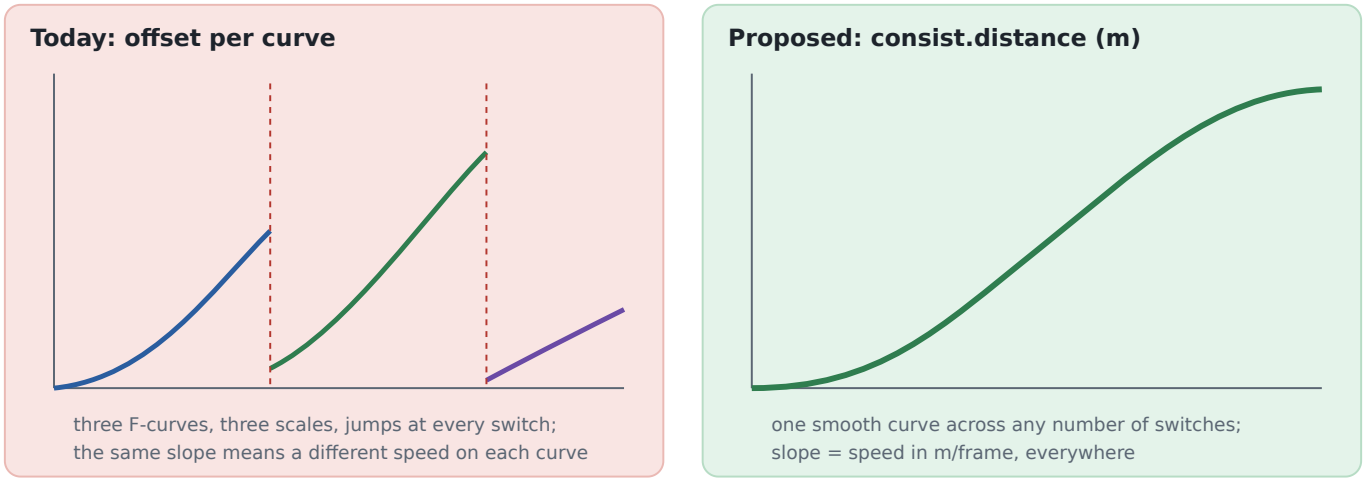


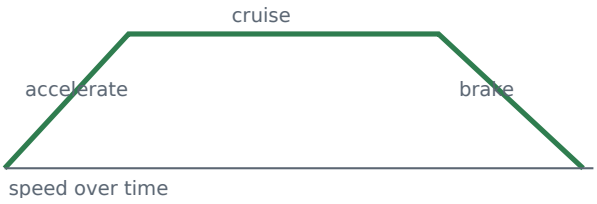
Figure 7.1 The same run keyed both ways. On the right, easing in the Graph Editor directly gives acceleration and braking.

Reversing is just a falling curve. A stop is a flat section. The route (Part 5) turns distance into curve and offset for every point of the train. **Migration**: the Convert operator (Part 5 §5.8) turns old offset keys into distance keys.

7.2 Timing tools

Speed Profile generator

Enter start frame, cruise speed (km/h or mph), acceleration and braking rates, and a stop distance or target marker. The operator writes distance keys with correct easing. It works for “depart the platform” shots.



Arrive At

Pick a target (a marker Empty on the track, a platform end, or another train's coupler). The operator computes the distance along the *route* to it and keys the train to stop exactly there on a chosen frame. It also handles **coupling moves** (stop with the couplers touching).

7.3 Live drive controls

Control	Status	Role in the finished add-on
Drag Slider	Keep	Proven, and the feel is right. Rewire it to nudge `consist.distance` (metres per second × power) instead of one curve's factor. Speed then feels the same on every curve.
Drag Pad	Recover	The 2D pad prototype (not in the latest file). It becomes the crash-safe “mouse drag”: an add-on-owned **modal operator** (not `TRANSFORM_OT_translate`), where horizontal mouse movement = distance. Writes happen inside the operator's own `modal()` event, never in a handler.
Mouse Drag	Remove	Delete once the Drag Pad works, since it is the only feature near the crash pattern.
Step buttons	New	“◀ 1 m / 1 m ▶” and “to next junction” for precise placement.
Snap to marker	New	Jump the train so its lead coupler (or any member) sits at a chosen track marker.

KEEP THE CRASH RULE

Drag Pad writes a single float per event and calls the pipeline directly. This is the same pattern as the Drag Slider, which has never crashed. It must never call `bpy.ops.transform.*`, and never rely on a depsgraph handler to do the writing.

7.4 Wheel modes

A wheel of radius *r* rolling a distance *s* without slipping turns by **θ = s ÷ r radians**. Each wheel group has its own diameter: a 2-4-2's drivers turn much more slowly than its truck wheels.



Figure 7.2 The rule behind Auto mode, and the one the legacy rig approximated with a tuned `prop1 / Maxtravel` driver.

Mode	Rotation =	Use for
AUTO	`facing × d_point / r` from a driver	Normal running. Zero effort, always in sync.
DIRECT	The animator's own keys (driver muted)	Full manual control of a wheel set.
AUTO + SLIP	`facing × d / r + slip_angle`, where `slip_angle` is a keyable bone property	Wheel slip on starting (drivers spin, loco barely moves). Key slip_angle rising fast while distance creeps.
AUTO × GRIP	`grip × (facing × d / r) + slip_angle`, with grip 0–1	Sliding or skidding under braking: grip → 0 locks the wheels while the train keeps moving.

```
# driver on WHL_D1_L rotation_euler[0]  (variables read custom properties on the rig)
#   d    = rig["ti_distance"]        written each frame by the pipeline (point distance)
#   r    = rig["ti_r_drivers"]       from the Vehicle Definition
#   g    = pose.bones["WHL_D1_L"]["grip"]         default 1.0 (keyable)
#   slip = pose.bones["WHL_D1_L"]["slip_angle"]   default 0.0 (keyable)
expression:  g * d / r + slip
```

Drivers are evaluated by Blender itself, so wheels also turn in background renders. Only the `ti_distance` value comes from Python, and a bake makes even that static (§7.8).

7.5 Secondary motion

These are all **additive** transforms on dedicated rig bones (“FX” bones under the body and bogies). They never feed back into the mechanism, and each has a master amount you can key to 0.

Effect	Model	Driven by
Body roll on curves	`roll = k_roll × v² × κ − cant`: lean outward when running faster than the cant was designed for	Speed v (m/s) and curvature κ = 1/R, both from the route at the body point
Hunting	`yaw = A(v) × sin(2π d / λ)`, a sinusoidal wander of bogies; λ ≈ 15–25 m	Distance d, with amplitude rising with speed
Rail-joint bounce	A small dip-and-recover every `joint_spacing` metres, front bogie then rear	d modulo spacing (the classic “clickety-clack”)
Suspension noise	Low-amplitude noise on body Z and roll	A Noise F-modifier on an FX bone, scaled by speed
Steam loco surge	Fore/aft pulse, 4 beats per driver revolution (2 cylinders, double-acting)	Driver wheel angle

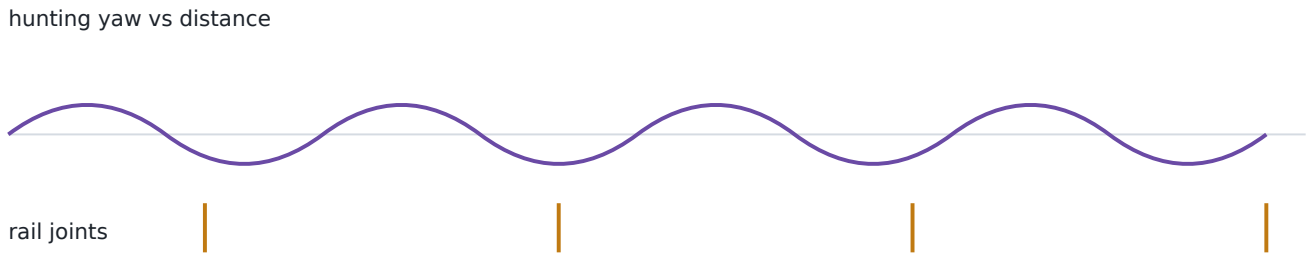


Figure 7.3 Distance-based effects stay perfectly repeatable: scrub back and forth and they don't change.

7.6 Speed readout

The Drive box shows live **speed (km/h or mph)**, **distance**, the **current segment**, and **next junction in N m**. It helps with matching real timetables and with hitting marks. A small viewport overlay (the same safe draw-handler technique as the existing HUD) can show it on-screen while scrubbing.

7.7 Render-safe evaluation

THE CURRENT RISK

Pivots, wagons and held offsets only update for the **active object**. Render an animation with nothing (or the camera) selected, and every pivot and wagon stays wherever it was last left, while the root alone follows its keys. This is audit issue #1, and it should be fixed before any production use.

1. **Evaluate all consists**
   In `frame_change_pre`, loop over `scene.track_it_consists` (not the selection) and run steps 1–4 of the pipeline (Part 3, Fig. 3.3).

2. **Mark the handler `@persistent`**
   That is already the case. Re-register on `load_post`, and rebuild caches.

3. **Keep the rest native**
   Constraints and drivers only after step 4, so the depsgraph evaluates everything in one pass.

4. **Test headless**
   `blender -b scene.blend -f 120` must produce the same pose as the viewport at frame 120 (Part 11 test plan).

7.8 Bake Train Motion

For render farms, exports (FBX/Alembic/USD to game engines) or sharing a file with someone who doesn't have the add-on:

- **Bake**: for a frame range, evaluate each frame and key every managed constraint's offset_factor and influence (CONSTANT on influence), plus `ti_distance` on rigs. Tag the result `ti_baked = True`.
- While baked, the pipeline **skips** that consist. Playback is pure keyframes.
- **Clear Bake**: remove those keys and return to live evaluation.
- Optional **Bake to Transforms**: `visual_transform_apply` per frame, for export to engines without Follow Path.

PART 8

Consist system

Trains made of real vehicles: built from the library, spaced by their couplers, driven from any car, split and joined on the timeline, and pushed backwards into sidings without any special cases.

8.1 What the prototype does

8.2 Target consist model

8.3 Spacing maths

8.4 Coupling and uncoupling

8.5 Driving from any car

8.6 Train Builder UI

8.1 What the prototype does

CURRENT STATE (LATEST SOURCE)

- **Consist** box: wagon list with **+/−**. Each wagon is a `copy()` of the root with its own Follow Path. Its offset is the gap to the car ahead (default copied from the previous wagon).
- **Wagon Data**: a `vehicle_root` field stub (“library not built yet, always clones the root”).
- The root keeps a **cumulative route distance** and a **history log** of curve changes (`distance, from/to curve, offsets, direction`). Junctions are solved geometrically so spacing doesn't drift by a metre at each switch.
- A trailing wagon that reaches a fork first (for example when reversing) decides it itself and writes it to the shared log.
- **Fix Train** reruns the sync three times. It also runs automatically when the Drag Slider goes idle. **Consist Debug** lists the log.

This is the right *idea* (distance-based, shared history, whichever end leads decides). Part 5's occupied path is the same idea with the history replaced by turnout states, which removes the dedupe/prune machinery. The prototype is a good test bed for checking the new evaluator: both should put every wagon in the same place on the same test scenes.

8.2 Target consist model



Every member is a full vehicle (own root, pivots, rig). The tender is its own vehicle, coupled with gap ≈ 0 by a drawbar.

Figure 8.1 A consist is an ordered list of vehicle instances plus one distance channel.

8.3 Spacing maths

With each vehicle's root at its centre and coupler distances in its definition (Part 6 §6.3), the reference distance of member *k* behind the anchor is:

```
d(0) = consist.distance
d(k) = d(k−1) − [ rear_coupler(k−1) + gap(k) + front_coupler(k) ]       (facing −1 swaps front/rear)

pivot point of member k:   d(k) + facing(k) × pivot.offset
```

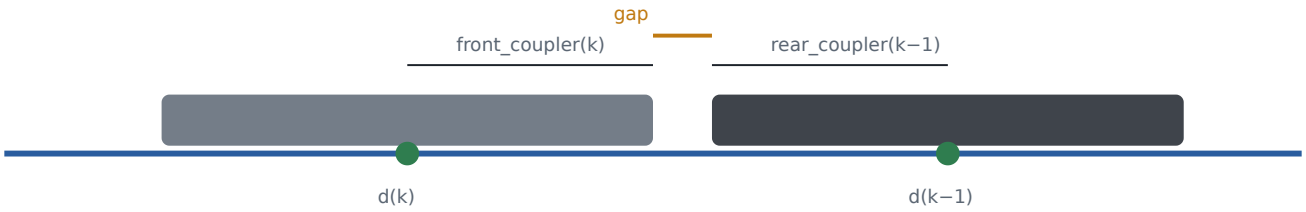


Figure 8.2 Spacing is measured **along the route**, so cars stay correctly spaced through curves and switches.

8.4 Coupling and uncoupling

From session 5: “If the user uncouples, the cars stay on the curve but don't react to dragging… maybe a decouple mode lets the drag slider on a car's own root work on its own.” As an animatable design:

Operation	What the add-on does
Uncouple at frame F (between member k−1 and k)	Creates consist *B* from members k… Its distance channel is keyed at F to the exact value that keeps each car where it is, then held flat (the cars stand still). Consist A keeps members 0…k−1. From F onwards the two move independently. Before F, the pipeline still treats them as one train (a “split event” stored on A: `split_frame`, `child_consist`).
Couple at frame F	Use **Arrive At** to bring the couplers together, then Couple. From F, B's members are appended to A with the measured gap, and B's distance channel is ignored after F.
Drag an uncoupled car	Selecting any car of B drives B. It has its own Drag Slider target, which is exactly the “decouple mode” idea.
Run-round / shunting	Just sequences of the above, with turnout keys (Part 5). The loco uncouples, runs round via the loop, and couples to the other end: facing is unchanged, but it becomes member 0 of a re-ordered list.

DECISION

Should split and join be **timeline events** (so scrubbing before F shows the joined train) or **destructive edits** (make two consists and animate each)? Recommendation: **timeline events**. Shunting scenes are the main reason to build consists, and they need to scrub correctly.

8.5 Driving from any car

- The Drive box acts on **the consist of the selected object**, whether that is a loco, wagon, rig bone or mesh. The add-on walks up from any selected part to its vehicle, then its consist.
- Pushing (the loco at the back) needs nothing special: when distance decreases, the leading end is the rear member (Part 5).
- **No collisions.** As agreed in session 5, running a train into itself or another train just clips. An optional overlay can flag overlap between consists on the same path segment in red, as a hint rather than a barrier.

8.6 Train Builder UI

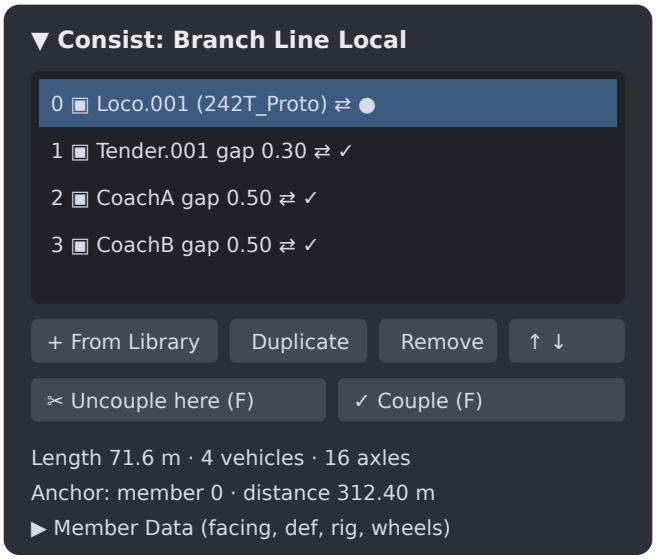


Figure 8.3 Mock-up of the finished Consist box (not built).

⇄ flips a vehicle's facing (runs backwards)

✓ = coupled to the car ahead (keyed state)

“From Library” opens the Asset Browser, filtered to Track It vehicles (Part 9)

Row order = train order; drag to reorder

The prototype's clone-the-root behaviour stays as “Duplicate” for quick proxies

Members are linked library overrides or local copies, so ten identical coaches share one mesh and rig.

PART 9 · WORKFLOW

From unrigged model to Track It asset

A step-by-step walkthrough of taking a plain, unrigged model (here a **four-wheel mine cart**) and turning it into a library vehicle you can drop onto any track, couple into a train, and animate. The steps marked Tool are proposed add-on features; everything else can be done by hand today.

9.1 The whole journey

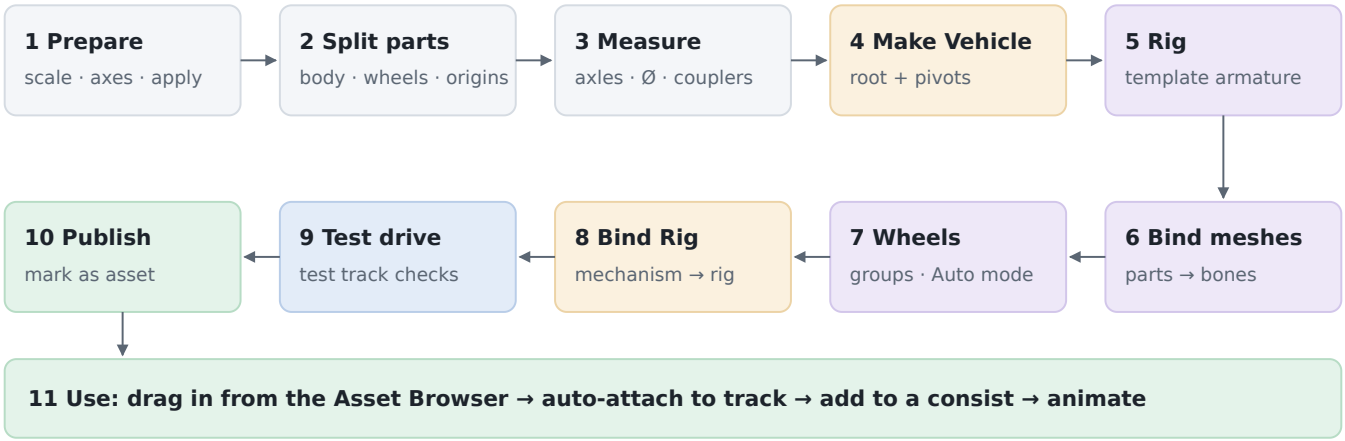
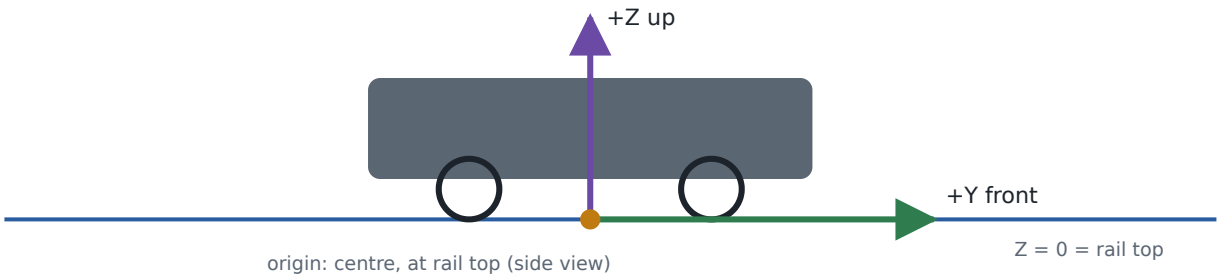


Figure 9.1 Colour follows the layer key: grey = model prep, amber = mechanism, violet = rig, blue = track, green = library.

9.2 Step by step

1 **Prepare the model**

- **Real-world scale in metres.** Check a known dimension (a mine cart's gauge is roughly 0.6 m; standard gauge is 1.435 m). Wheel rotation and cant maths depend on it.
- **Orientation:** front of the vehicle toward **+Y**, up **+Z**. This matches Follow Path's `FORWARD_Y / UP_Z`, which Track It always uses.
- **Apply** rotation and scale on every part (Ctrl+A). Leftover object scale breaks bone placement and wheel diameters.
- **Rail-top origin:** the model should sit so its wheel treads touch Z = 0. That plane is where the curve will be.



2 **Split into parts and set their origins**

Each thing that moves on its own must be its own object, with its **origin at its rotation centre**:

Part	Origin at	Name (convention)
Body / tub	vehicle centre, rail top	`MC_Body`
Each wheelset (or wheel)	axle centre	`MC_Wheel_F`, `MC_Wheel_R` (or `_F_L`, `_F_R`)
Frame / chassis	vehicle centre	`MC_Frame`
Couplers, doors, tipping hinge…	their hinge	`MC_Coupler_F`, `MC_Door`

Tip: in Edit Mode select the axle's edge loop, Shift+S → Cursor to Selected, then in Object Mode use Set Origin → Origin to 3D Cursor. These are the same manual steps the early sessions used.

3 Measure `Tool: Measure Vehicle`

With the parts named, a **Measure Vehicle** operator can read everything from bounding boxes: axle Y positions (from wheel origins), wheel diameters (bounding-box height), overall length, and coupler tips (the furthest ±Y points of the coupler objects). By hand, read each wheel's Location Y and Dimensions Z in the N-panel.

```
Mine cart (example)     axle F  +0.45 m   axle R  -0.45 m   wheel Ø 0.36 m
                        front coupler +0.95 m   rear coupler -0.95 m   length 1.90 m
```

4 Make Vehicle `Tool: wizard`

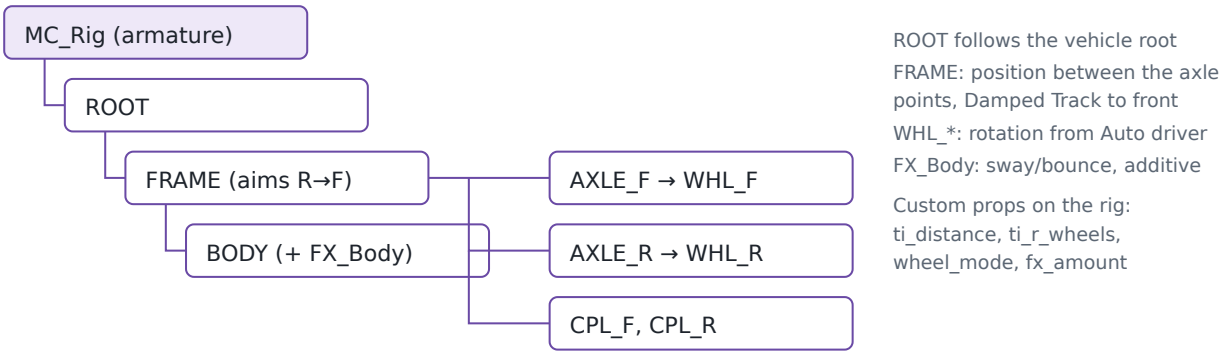
Select all the parts and run **Make Vehicle**, then pick a template:

Template	Creates
Rigid 2-axle (mine cart, wagon)	Root + 2 AXLE_POINT pivots at the measured axle offsets
Bogie car	Root + 2 TRACKED pivots at the bogie centres
Steam loco (Whyte n-n-n)	Root + drivers as AXLE_POINTs + RADIAL trucks (Part 10)
Custom	Root only. Add pivots by hand, as in today's Vehicle box

Today's equivalent is to add an Empty (or cylinder) as root at the vehicle origin, then use Vehicle → **+** twice with offsets **+0.45** and **−0.45**.

5 Generate the control rig `Tool`

The template builds an armature with standard bone names, placed at the part origins:



6 Bind the meshes to the rig

Parent each part to its bone (select part, Shift-select rig, Pose Mode, select bone, Ctrl+P → **Bone**). The tool does this automatically by name: `MC_Wheel_F` → `WHL_F`, and so on. Meshes **never** parent to the mechanism root or pivots (the “never bolted” rule).

7 **Wheel groups** Tool

Group wheels that share a diameter: here one group, “wheels”, Ø 0.36 m, mode AUTO. The tool adds the `d / r` driver (Part 7 §7.4) to each wheel bone's X rotation and the `grip` and `slip_angle` properties for later.

---

8 **Bind Rig**

Fill Pivot Data (today's stub): pivot 1 → `MC_Rig` / `AXLE_F`, pivot 2 → `AXLE_R`, then **Bind Rig**. This creates the `TI_` constraints of Part 6 §6.6. After binding, hide the pivot Empties.

---

9 **Test drive** Tool: test track

The add-on ships a test scene: an S-curve at the minimum radius, a banked curve, a turnout and a reverse loop. **Test Drive** runs the vehicle through it and reports:

- Pivots stay on the rails (distance to curve < 1 mm) and the wheelbase is held (±1 mm).
- Wheel travel matches the rotation: a marker on the tyre shouldn't slide along the rail.
- Body overhang on the minimum radius (this sets the clearance you need).
- No mesh parented to mechanism objects; no unapplied scale; all bones bound.

---

10 **Publish to the library**

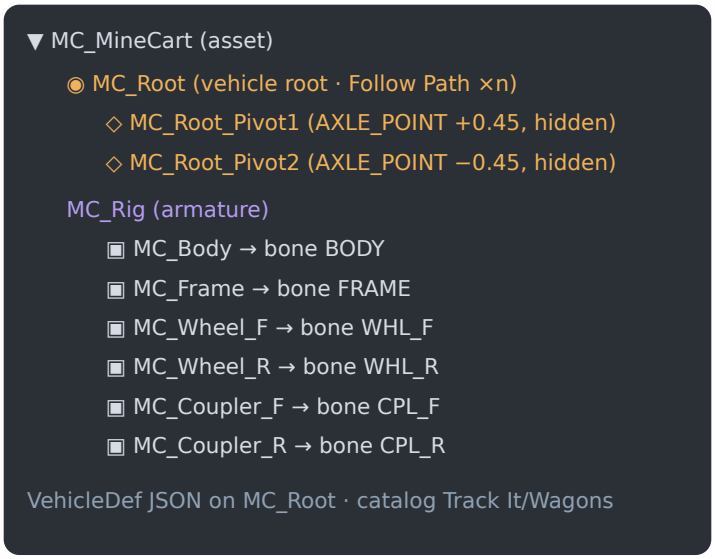
- Put root, pivots, rig and meshes in one collection, `MC_MineCart`. Right-click → **Mark as Asset**.
- Catalog: `Track It / Wagons`. Set a preview (render or auto).
- The Vehicle Definition (Part 6 §6.8) is stored on the root, so the asset carries everything.
- Save in a library .blend registered under Preferences → File Paths → Asset Libraries.

---

11 **Use it**

Drag the asset into a scene. The add-on detects a Track It vehicle and offers **Attach to nearest track** (the existing Attach logic). Add it to a consist from the Train Builder (**+ From Library**), then animate the consist's distance.

9.3 What the finished asset looks like



Checklist before publishing

- ☐ scale applied, metres, +Y forward
- ☐ wheels' origins on axle centres
- ☐ pivots match the measured axles
- ☐ every mesh parented to a rig bone
- ☐ no mesh parented to mechanism
- ☐ wheel groups + diameters set
- ☐ Bind Rig done, Empties hidden
- ☐ Test Drive passes
- ☐ couplers measured (front/rear)
- ☐ preview + catalog set

Track Doctor's vehicle checks automate most of this list.

Figure 9.2 Outliner view of the finished mine cart asset.

PROXY FIRST, FINAL MODEL LATER

Because meshes only parent to rig bones, you can publish a vehicle built from **proxy blocks** (as planned for the 2-4-2), animate whole scenes with it, and later swap in the finished model by re-parenting its parts to the same bones. The animation doesn't change at all.

PART 10 · CONCEPT STUDY

Rigging a 2-4-2 steam locomotive with a two-bogie tender

The hardest vehicle in the plan, from the first vehicle-architecture session: taking an **unrigged** 2-4-2 and its tender, deciding what tracks the rail and what is rigid, building the rig (trucks, drivers, rods, valve gear), and bringing both into the Track It framework as a coupled pair.

10.1 Meet the locomotive
10.2 Loco and tender: one vehicle or two?
10.3 Mechanism plan
10.4 How the frame is solved
10.5 Pony and trailing trucks
10.6 Wheels and quartering
10.7 Rods and valve gear
10.8 The tender
10.9 Complete bone hierarchy
10.10 Walkthrough: unrigged → in the framework
10.11 Pitfalls and checks
10.12 What this study adds to the backlog

All dimensions are an **illustrative prototype** chosen to be plausible for a small standard-gauge engine. Measure your own model (Part 9, step 3).

10.1 Meet the locomotive

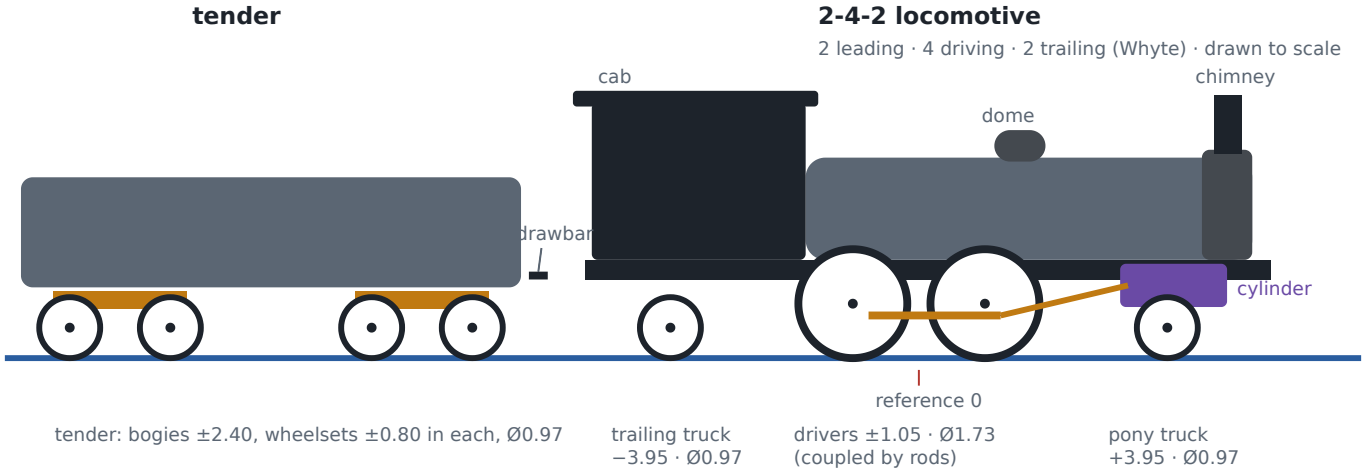


Figure 10.1 Side elevation, front to the right. Only four parts touch the rail independently on the loco: the two trucks and the two driving axles. The rest hangs off the frame.

Loco dimension (illustrative)	Value	Measured from the loco reference (centre of driving wheelbase)
Leading truck axle	+3.95 m	Pony truck pin (on frame): +2.60 m → radius arm 1.35 m
Driving axle 1 / 2	±1.05 m	Rigid driving wheelbase 2.10 m
Trailing truck axle	−3.95 m	Pin −2.60 m
Couplers front / rear	+5.90 / −5.90 m	Rear = drawbar pin to the tender
Crank radius / main rod	0.30 / 2.40 m	Stroke 0.60 m; the crosshead travels 2.10–2.70 m from the crank centre

10.2 Loco and tender: one vehicle or two?

Two vehicles, coupled ✓ (recommended)

- The tender swings on its own drawbar, as in reality.
- It reuses the ordinary bogie-car template.
- Tender-first running, or uncoupling for servicing, comes for free (Part 8).
- The pair can be published as a **consist preset** asset, so it drops in as a unit.

One vehicle with six pivots

- Simpler outliner.
- But the tender's body then has to be solved from pivots inside a loco-vehicle. That means a second body, which the data model doesn't have.
- Uncoupling becomes impossible.

10.3 Mechanism plan

Vehicle	Pivot	Role	Offset	Drives rig bone
Loco	LEAD_TRUCK	RADIAL	+3.95	TRUCK_F (aims at pin +2.60 on FRAME)
	DRIVER_1	AXLE_POINT	+1.05	FRAME (front end of chord)
	DRIVER_2	AXLE_POINT	−1.05	FRAME (rear end of chord)
	TRAIL_TRUCK	RADIAL	−3.95	TRUCK_R (aims at pin −2.60)
Tender	BOGIE_F	TRACKED	+2.40	T_BOGIE_F
	BOGIE_R	TRACKED	−2.40	T_BOGIE_R

Consist: `[Loco (facing +1), Tender (gap 0.30 m)]`. Tender couplers ±4.10 m.

10.4 How the frame is solved

On a real engine the **rigid driving wheelbase** sets the frame's line; the trucks swing underneath. So the frame is placed on the **chord** between the two driver axle points (the same technique as the coach body in Part 6 §6.4):

> Beyond the chord ends the rail curves **inward**, so each truck axle sits inside the frame line and the truck swings. On R = 120 m that is 0.06 m at the axle, a truck angle of about 1.6° (curve exaggerated here).

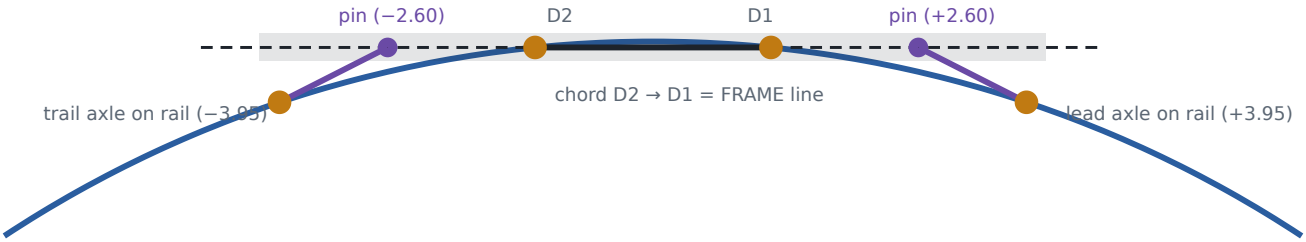


Figure 10.2 Plan view on a curve. FRAME: Copy Location D1 (1.0) + D2 (0.5) → midpoint; Damped Track → D1; banking averaged from D1 and D2.

10.5 Pony and trailing trucks

A pony truck is an axle on a **radius arm** pivoting on a pin fixed to the frame. The rig reproduces exactly that:

```
TRUCK_F   (child of nothing; driven entirely by constraints)
  Copy Location  → LEAD_TRUCK Empty           axle stays on the rail
  Damped Track   → FRAME, head/tail offset to the pin (+2.60)   arm points at the pin
                   (or: a PIN_F bone, child of FRAME at +2.60, as the Damped Track target)
  Copy Rotation  → LEAD_TRUCK Empty, roll only           takes the rail's cant
WHL_F_L / WHL_F_R   (children of TRUCK_F) : rotation from Auto driver, Ø 0.97 m
```

WHY NOT JUST COPY TRANSFORMS FROM THE EMPTY?

The Empty's heading is the **rail tangent** at the axle, which is correct for the wheelset but not for the truck frame. The truck frame must point at its pin, otherwise the radius arm visibly detaches from the loco frame on curves. Use the Empty for position and roll, and the pin for yaw.

10.6 Wheels and quartering

- **Two wheel groups**: drivers (Ø 1.73 m, r = 0.865 m) and trucks (Ø 0.97 m, r = 0.485 m). Same distance, different rotation rates, both from the Auto driver `d / r`.
- All four drivers share **one angle** (they are coupled by rods). Drive DRIVERS_ANGLE once, and let each driver bone copy it, so a slip animation spins them together.
- **Quartering**: the right-hand crank pins sit 90° ahead of the left-hand ones (a common convention, check your prototype). Build it into the crank pin placement (the right side bone rotated +90°), not

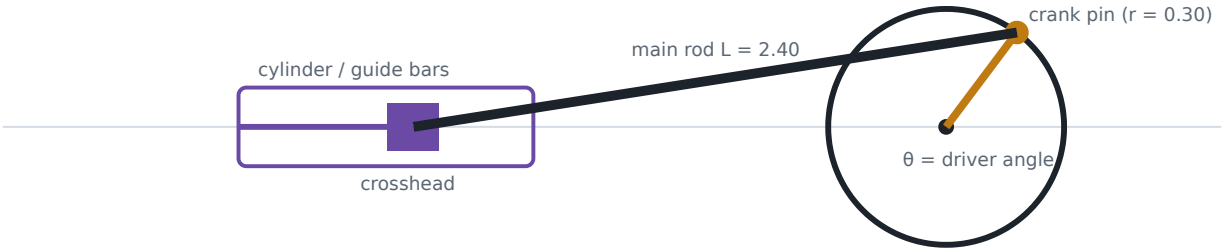
into the drivers, so wheel rotation stays a single value.

10.7 Rods and valve gear

Coupling rods

The coupling rod joins the crank pins of D1 and D2. Because both pins turn together, the rod **translates without rotating**: bone `ROD_CPL_L` = child of FRAME, **Copy Location** from `CRANK_D1_L`. It stays parallel to the frame automatically.

Main (connecting) rod: slider-crank



crosshead x = r·cos θ + √(L² − (r·sin θ)²) → 2.70 m (θ=0) … 2.10 m (θ=180°)

Figure 10.3 Put this formula in a driver on the crosshead bone's local Y. It is exact, with no IK solver to wobble. The main rod bone sits at the crosshead and **Damped Tracks** the crank pin; the piston rod is a child of the crosshead.

Walschaerts valve gear (optional depth)

Part	Rigging approach
Return crank / eccentric rod	Return crank on the crank pin (fixed offset angle ~90°). Eccentric rod: Damped Track from the return-crank end to the expansion link's lower end.
Expansion link	Rocks about a fixed trunnion on the frame. Its angle comes from the eccentric rod end (Damped Track, or a Transform constraint).
Radius rod + die block	The die position in the link is the **reverser**: a custom property `reverser` (−1 … +1) that the animator keys for forward/back gear and cut-off. The radius rod runs from the die to the combination lever.
Combination lever / union link	Hangs from the radius rod; its bottom follows the crosshead via the union link. Two-bone IK with a pole works well here.

MAKE REVERSING VISIBLE

Tie `reverser`'s default to the sign of speed with a driver: `sign(Δd)`, smoothed. The valve gear then flips into back gear when the train reverses, and the animator can override it. Few rigs bother with this, and viewers notice.

10.8 The tender

- An ordinary **bogie car**: two TRACKED pivots at ±2.40 m; the body on the chord (Part 6 §6.4).
- Each bogie bone takes **Copy Transforms** from its pivot Empty. Its two wheelsets are children at ±0.80 m, rotation Auto, Ø 0.97 m.
- **Drawbar**: a bone on the loco's rear coupler with **Damped Track** to a target bone on the tender's front coupler. Its length change on curves is tiny, and a **Stretch To** set to volume “none” hides it if needed.
- Fall-plate and coal/water details are children of the tender BODY. Add some sway (FX) that is slightly out of phase with the loco for realism.

10.9 Complete bone hierarchy

LOCO_242 rig

ROOT ← vehicle root
- FRAME ← chord(D1, D2), Damped Track D1
  - BODY − FX_BODY (sway, bounce)
    - boiler · smokebox · cab · dome (meshes)
  - PIN_F (+2.60) PIN_R (−2.60)
  - AXLE_D1 − WHL_D1_L, WHL_D1_R − CRANK_D1_L/R
  - AXLE_D2 − WHL_D2_L, WHL_D2_R − CRANK_D2_L/R
  - ROD_CPL_L / R (Copy Loc ← CRANK_D1)
  - CYL_L / R − XHEAD_L/R (driver) − PISTON_L/R
    - ROD_MAIN_L/R (at XHEAD, Damped Track → CRANK_D2*)
  - VALVE_L/R: RETCRANK · ECC_ROD · LINK · RADIUS · COMB
  - CPL_F · CPL_R · DRAWBAR (Damped Track → tender)
- TRUCK_F ← LEAD_TRUCK Empty, Damped Track PIN_F
  - WHL_F_L, WHL_F_R
- TRUCK_R ← TRAIL_TRUCK Empty, Damped Track PIN_R
  - WHL_T_L, WHL_T_R

Custom props: ti_distance · ti_r_drivers 0.865 · ti_r_trucks 0.485 · reverser · fx_amount drivers_angle · grip · slip_angle (Part 7 §7.4)

TENDER rig

ROOT ← tender root
- T_BODY ← chord(BOGIE_F, R)
  - FX_T_BODY · coal · tank
- T_BOGIE_F ← BOGIE_F Empty
  - WHL_TF1/TF2 (L, R)
- T_BOGIE_R ← BOGIE_R Empty
  - WHL_TR1/TR2 (L, R)

* the main rod drives whichever axle the prototype uses (often the rear driver on a 2-4-2). Check the model.

Colour key: FRAME and trucks are driven by the mechanism; everything else is ordinary rig the artist can key.

Figure 10.4 Suggested bone names. The Make Vehicle “Steam loco” template (Part 9) would generate everything above except the valve gear, which is a separate “Add Walschaerts gear” helper.

10.10 Walkthrough: unrigged → in the framework

1. **Prepare both models**
   Metres, +Y forward (loco chimney toward +Y), apply transforms, rail top at Z = 0. Keep the loco and tender as separate collections.

2 **Split the loco into parts**
Frame, boiler, cab, each wheel (8), crank pins, coupling rods (2), main rods (2), crossheads (2), piston rods (2), valve gear parts, and truck frames (2). Set each origin at its rotation centre (wheels at the axle, rods at their pin ends, crosshead at its slide centre).

3 **Measure**
Run **Measure Vehicle**, or read origins by hand: axle Ys (+3.95, +1.05, −1.05, −3.95), wheel diameters (1.73 / 0.97), and the pin positions of the truck arms.

4 **Make Vehicle → “Steam loco”**
Enter the Whyte notation `2-4-2`. The template creates the root at the centre of the driving wheelbase, two AXLE_POINT pivots at the drivers, two RADIAL pivots at the truck axles with pin offsets, and the rig from 10.9 (without valve gear). Check the gizmos in the viewport.

5 **Rods**
Add the crosshead drivers (Fig. 10.3), the coupling rod Copy Locations and the quartering on the crank bones. Then optionally **Add Walschaerts gear** and wire `reverser`.

6 **Bind meshes and rig**
Name-based auto parenting, then **Bind Rig**. Hide the pivot Empties.

7 **Tender**
Make Vehicle → “Bogie car” with bogies at ±2.40 m, wheelsets at ±0.80 m within each bogie, Ø 0.97 m. Bind.

8 **Couple and publish**
Create a consist `[Loco, Tender gap 0.30]`. Publish the loco and tender as vehicle assets, and the pair as a **consist preset** asset (`Track It / Consists / 242T + Tender`).

9 **Test drive**
Run it on the test track: minimum radius (check the truck-to-cylinder clearance), the banked curve (average of cant across the drivers), the turnout (the trucks swing through independently), a reverse run (valve gear flips, rods run backwards), and tender-first running (flip the loco's facing and move the tender to the front of the consist).

10 **Animate a shot**
Key the turnout at the station throat, then the consist's distance: depart, accelerate, cruise and brake with the Speed Profile tool. Add a wheel-slip burst on starting (slip_angle keys on DRIVERS_ANGLE with grip at 1), and tune FX amounts.

10.11 Pitfalls and checks

Symptom	Cause and fix
Pony truck wheels drift off the rail on curves	The truck was given Copy Transforms from the Empty *and* is parented to FRAME. Remove the parent: trucks are driven by constraints only.
The body cuts the inside of curves	The body is following the root instead of the driver chord. Use FRAME from 10.4.
Rods jitter or flip at the dead centres	IK used for the main rod. Use the exact slider-crank driver.
Wheels skid visually	Wrong diameter group, or the model wasn't in metres. Re-measure; check with the tyre-marker test.
The tender lags or leads by a few cm after a switch	Legacy odometer drift (today's consist prototype). Fixed by the occupied-path evaluator (Part 5).
Rods turn backwards when reversing	They shouldn't: they follow `d`, which decreases. If they do, a driver uses `abs()`, or facing was applied twice.

10.12 What this study adds to the backlog

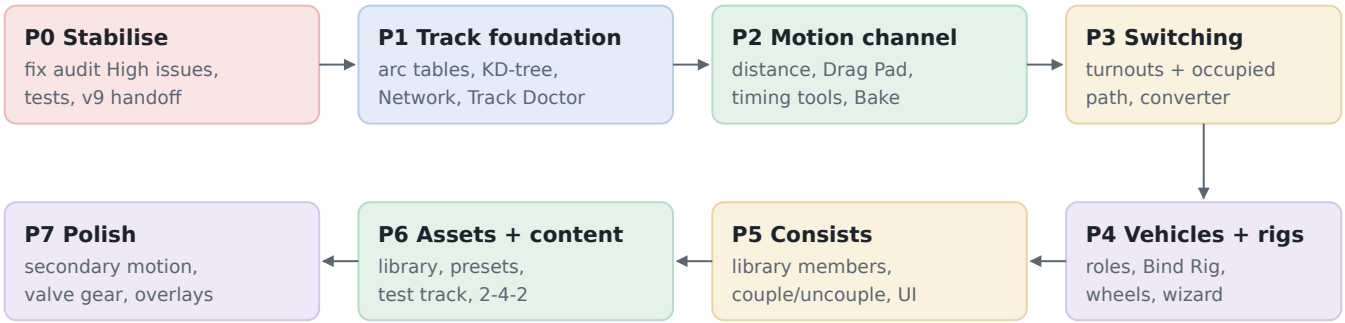
- Pivot roles (RADIAL with `pin_offset`, AXLE_POINT) in `TrackIt_VehiclePivotItem`.
- A FRAME/BODY chord solver generated by Bind Rig.
- Multiple wheel groups per vehicle, and a shared driver angle.
- Slider-crank and Walschaerts helper generators, and the `reverser` convention.
- Consist presets as assets.
- Test-track clearance checks for truck swing.

PART 11

Roadmap to completion

Every Part of this guide turned into ordered phases, each shippable and each keeping the add-on usable. It also covers the test plan that stops regressions, the decisions still open, and the first five things to do next session.

11.1 Phases at a glance



What you can do after each phase

P0: render any scene reliably; edit curves without stale offsets; two trains in one scene.
P1: build big layouts that stay fast; switches detected once; yard sidings never confused.
P2: animate a whole run with one F-curve in metres; drag with the mouse safely; farm-ready bakes.
P3: set points, shunt, switchback, back into sidings from the last car, with two trains sharing turnouts.
P4–P7: any model → vehicle in minutes; real trains from the library; the 2-4-2 with working rods.

Figure 11.1 Phase order. P4 (vehicle roles, Bind Rig) can start alongside P2/P3 if you want rigging progress sooner. It only depends on P0.

11.2 Backlog by phase

Phase	Item	Ref	Size
P0	Evaluate pivots and wagons for **all** vehicles, not only the active object	Audit #1, Part 7 §7.7	M
P0	Arc cache keyed by object and geometry hash; invalidate on edit	Audit #2, Part 4 §4.2	S
P0	Clear inherited collections on wagon clones (verify first)	Audit #4	S
P0	Pivots resolve via the existing route resolver (crossing fix)	Audit #3, Part 6 §6.5	S

Phase	Item	Ref	Size
P0	bl_info version bump, minimum Blender version, “Migrate constraints” operator	Part 1 §1.4, audit #12	S
P0	Recover the Drag Pad prototype from the fork; write the v9 handoff	Part 1 §1.4	S
P0	Headless test harness plus five baseline scenes	§11.3	M
P1	Arc tables from `interpolate_bezier`, cyclic support, KD-tree nearest	Part 4 §4.2	M
P1	Track Network + Scan Track (joints, turnouts with span test, dead ends)	Part 4 §4.3	L
P1	Track Doctor; joint auto-continue toggle; Auto Cant	Part 4 §4.5–4.8	M
P1	Split into package / Extension; UUIDs on managed objects	Part 3 §3.5, audit #8	M
P2	Consist distance as the Motion channel (a lone vehicle = consist of one)	Part 7 §7.1	L
P2	Drag Slider on distance; Drag Pad modal; step buttons; speed readout	Part 7 §7.3, 7.6	M
P2	Speed Profile, Arrive At; Bake / Clear Bake	Part 7 §7.2, 7.8	M
P3	Junction objects with keyframeable state; Junctions Ahead panel	Part 5 §5.5, 5.7	M
P3	Occupied-path evaluator with per-frame cache	Part 5 §5.5	L
P3	Convert Keyed Switches; retire keyframe mining, shadow sync, odometer log	Part 5 §5.8	M
P4	Pivot roles; chord FRAME/BODY solver; Bind/Unbind Rig	Part 6	L
P4	Wheel groups, Auto/Direct/Slip/Grip drivers; facing	Part 7 §7.4, Part 6 §6.7	M
P4	VehicleDef, Measure Vehicle, Make Vehicle wizard with templates	Part 9	L
P5	Library-based members, coupler spacing, couple/uncouple events, Train Builder UI	Part 8	L
P6	Asset library layout, test track scene, track presets, mine cart, coach, 2-4-2 + tender	Parts 4, 9, 10	L
P7	Secondary motion FX, Walschaerts helper, reverser, overlays, performance pass	Part 7 §7.5, Part 10	M

Size: S ≈ one session, M ≈ 2–3 sessions, L ≈ 4+ sessions, at the pace of the sessions so far.

11.3 Test plan

Most of the bugs found so far (the flag-ordering race, contaminated keys, stale matrix reads, the keyframe-removal iterator) were caught by hand, after the fact. A small automated suite running Blender headless catches them before you open Blender:

```
blender -b tests/scenes/switch_basic.blend -P tests/run.py -- --case all
```

Scene	Assertions
attach_roundtrip	Attach → Detach → Attach leaves the world matrix unchanged (±1e-5), with no rotation stacking.
arc_accuracy	For 20 factors, the add-on's predicted position = Blender's evaluated Follow Path position (±1 mm), including a cyclic curve and resolution 4.
switch_basic	Diverge at frame 36: positions at frames 1/35/36/37/120 match stored golden values when scrubbed forward, backward and randomly.
switchback	Main → curve 2 → reverse onto curve 3. Every pivot and wagon is on the expected curve at each checkpoint.
yard_sidings	Dead-end sidings ending close together never produce a switch (the §8.5 regression).
consist_spacing	Four wagons through two switches: the gap between consecutive members along the route stays within ±5 mm.
render_parity	With nothing selected, a background render pose at frames 50 and 120 equals the viewport pose.
edit_invalidation	Move a curve point after attaching: the next evaluation uses the new geometry.

11.4 Decisions still open

Question	Recommendation	Where
Adopt turnout state + occupied path instead of per-vehicle history?	Yes, in Phase 3, via the converter	Part 5
Per-train Route Plan overrides?	Optional, off by default	Part 5 §5.6
Live drag reads the current or the next keyed turnout state?	A setting; default “current”	Part 5 §5.6
Animate distance instead of offset_factor?	Yes	Part 7 §7.1
Vehicle root at the centre or the front coupler?	Centre, plus coupler distances	Part 6 §6.3
Turnout toe: auto-detected or a marker Empty?	Auto, with an optional override	Part 4 §4.4
Loco and tender: one vehicle or two?	Two, coupled, plus a consist preset	Part 10 §10.2
Couple/uncouple as timeline events or destructive edits?	Timeline events	Part 8 §8.4

Question	Recommendation	Where
Keep Mouse Drag at all once the Drag Pad works?	Remove it	Part 7 §7.3
Joint auto-continue: scene-wide or per vehicle?	Per junction now, per vehicle later	Part 4 §4.5

11.5 Risks

Risk	Mitigation
The modal-transform crash comes back	No handler writes during `TRANSFORM_OT_*` modals (check `window.modal_operators`); live control only through add-on-owned widgets; a test that runs a scripted grab.
Performance on large layouts	Network built once; KD-trees; per-frame route cache; profile with 50 curves and 5 trains as an acceptance target.
Blender API changes (layered Actions changed in 4.4/5.x)	Isolate F-curve access in one helper (`_track_it_find_fcurve` already does this); pin the minimum version in the manifest.
A big rewrite stalls progress	Every phase ships. The old paths stay behind toggles until the converter and tests prove the new ones.
Scenes from before the migration	Migrate/Convert operators, plus Bake as an escape hatch.

11.6 Next five actions

1. **Write the v9 handoff and bump the version**
   Record the consist prototype (not in v8), the missing Drag Pad, and this guide's audit list.

2. **Fix “active object only”**
   Loop over every object with a Track It root in both handlers. This is the smallest change with the biggest effect (renders).

3. **Fix the arc cache**
   Key it by `curve.as_pointer()` plus a geometry hash, and clear it in `depsgraph_update_post` for updated curve ids.

4. **Route pivots through `_track_it_resolve_route_position`**
   This closes v8 §3.3 with code that already exists.

5 **Build the headless test harness**

Start with attach_roundtrip, arc_accuracy and render_parity, then refactor with confidence.

CLOSING NOTE

The hard, fiddly parts are already solved: arc-length attach, crash-safe live driving, influence switching, junction-exact consist spacing. What's left is mostly **moving decisions to where they belong** (switches on the track, motion on the train, looks on the rig) and building the tools around them. The design sessions already pointed that way.